



Universidad Internacional de La Rioja
Escuela Superior de Ingeniería y Tecnología

Máster Universitario en Análisis y Visualización de Datos
Masivos/ Visual Analytics and Big Data

Diseño e implementación de una arquitectura de datos masivos para el análisis predictivo y la visualización interactiva

Trabajo fin de estudio presentado por:	Laura Carrasco Mateos
Tipo de trabajo:	Desarrollo de software
Director/a:	Dr. Luis Miguel Fernández-Galván
Fecha:	Septiembre de 2026

Resumen

Este Trabajo Fin de Máster tiene como objetivo diseñar e implementar una arquitectura de datos masivos orientada al análisis predictivo y a la visualización interactiva. Para validar la propuesta se ha desarrollado un caso práctico basado en datos abiertos de tráfico urbano de la ciudad de Madrid, utilizando información histórica de intensidad de tráfico correspondiente a los años 2023 y 2024 y datos de ubicación de los puntos de medida publicados por el Ayuntamiento de Madrid. La arquitectura implementada integra fases de ingesta, limpieza, transformación, construcción de un dataset analítico, entrenamiento de modelos predictivos y explotación visual mediante un cuadro de mando interactivo en Power BI.

El proceso de ingesta se ha aplicado sobre 24 ficheros mensuales en formato CSV, con un volumen inicial de 308.458.911 registros. Para construir un prototipo funcional y computacionalmente manejable se seleccionaron los 50 puntos de medida con mayor número de registros válidos. Tras aplicar los procesos de filtrado, conversión de tipos, eliminación de registros erróneos y agregación horaria, se obtuvo un dataset final de 836.228 registros horarios. Posteriormente, se formuló un problema de regresión supervisada cuyo objetivo fue predecir la intensidad horaria del tráfico por punto de medida. Se comparó un modelo base, basado en la intensidad registrada en la misma hora del día anterior, con un modelo Random Forest Regressor.

Los resultados obtenidos muestran que el modelo Random Forest mejora de forma clara al modelo base, reduciendo el MAE de 433,41 a 164,16 vehículos/hora, el RMSE de 840,93 a 311,83 y aumentando el coeficiente R^2 de 0,7569 a 0,9666. Finalmente, los datos limpios, las predicciones, las métricas y las variables explicativas se integraron en un dashboard interactivo compuesto por páginas de calidad de datos, análisis temporal, análisis geográfico y evaluación del modelo predictivo. La arquitectura desarrollada demuestra la viabilidad de integrar datos abiertos, procesamiento analítico, aprendizaje automático y visualización interactiva en una solución modular, reutilizable y orientada a la toma de decisiones.

Palabras clave: datos masivos, análisis predictivo, movilidad urbana, tráfico, Power BI, Random Forest, arquitectura de datos, visualización interactiva.

Palabras clave: datos masivos, análisis predictivo, movilidad urbana, Power BI, visualización interactiva.

Abstract

This Master's Thesis aims to design and implement a big data architecture focused on predictive analytics and interactive visualization. The proposal is validated through a practical case study based on open urban traffic data from the city of Madrid, using historical traffic intensity records from 2023 and 2024 together with the geolocation dataset of traffic measurement points published by the Madrid City Council. The implemented architecture integrates data ingestion, cleaning, transformation, analytical dataset construction, predictive model training and visual exploitation through an interactive dashboard in Power BI.

The ingestion process was applied to 24 monthly CSV files, with an initial volume of 308,458,911 records. In order to build a functional and computationally manageable prototype, the 50 measurement points with the highest number of valid records were selected. After filtering errors, converting data types, removing invalid records and aggregating the data at hourly level, a final clean dataset of 836,228 hourly records was obtained. A supervised regression problem was then defined, with the aim of predicting hourly traffic intensity by measurement point. A baseline model based on the intensity observed at the same hour of the previous day was compared with a Random Forest Regressor.

The results show that the Random Forest model clearly improves the baseline model, reducing the MAE from 433.41 to 164.16 vehicles/hour, the RMSE from 840.93 to 311.83, and increasing the R^2 coefficient from 0.7569 to 0.9666. Finally, the cleaned data, predictions, evaluation metrics and explanatory variables were integrated into an interactive dashboard composed of data quality, temporal analysis, geographical analysis and predictive model evaluation pages. The developed architecture demonstrates the feasibility of integrating open data, analytical processing, machine learning and interactive visualization into a modular and reusable solution aimed at supporting data-driven decision-making.

Keywords: big data, predictive analytics, urban mobility, traffic, Power BI, Random Forest, data architecture, interactive visualization.

Keywords: : Biga Data, predictive analytics, urban mobility, Power BI, interactive visualization.

Índice de contenidos

1.	Introducción	1
1.1.	Motivación	3
1.2.	Planteamiento del trabajo	4
1.3.	Estructura del trabajo	6
2.	Contexto y estado del arte	8
2.1.	Contexto del problema	8
2.2.	Estado del arte	9
2.2.1.	Arquitecturas Big Data para el análisis de datos masivos	9
2.2.2.	Análisis predictivo aplicado a la movilidad urbana	12
2.2.3.	Visualización interactiva de datos de movilidad	14
2.2.4.	Herramientas y soluciones existentes	15
2.3.	Conclusiones	17
3.	Objetivos concretos y metodología de trabajo	19
3.1.	Objetivo general	19
3.2.	Objetivos específicos	20
3.3.	Metodología del trabajo	21
3.3.1.	Enfoque metodológico	21
3.3.2.	Fases del desarrollo	22
3.3.3.	Trazabilidad y control de la evaluación	25
3.4.	Uso de herramientas de inteligencia artificial	26
4.	Marco normativo	27
4.1.	Protección de datos personales	27
4.2.	Reutilización de información del sector público	28
4.3.	Normativa relacionada con movilidad	29

4.4.	Consideraciones éticas y de seguridad.....	29
4.5.	Aplicación al TFM.....	30
5.	Desarrollo específico de la contribución.....	32
5.1.	Descripción general de la arquitectura propuesta.....	32
5.2.	Instanciación de la arquitectura en el caso de la movilidad urbana.....	34
5.3.	Tecnologías seleccionadas y justificación.....	35
5.4.	Fuentes de datos utilizadas	37
5.5.	Proceso de ingesta y caracterización de los datos	38
5.6.	Limpieza, transformación y agregación horaria	39
5.7.	Construcción del dataset analítico	41
5.8.	Modelado predictivo de intensidad del tráfico	42
5.9.	Estrategia de entrenamiento y evaluación	43
5.10.	Importancia de las variables.....	44
5.11.	Análisis complementario de los errores.....	45
5.12.	Diseño del informe interactivo en Power BI	46
5.13.	Reutilización y posibilidades de ampliación	47
6.	Código fuente y datos analizados	48
6.1.	Organización del repositorio	48
6.2.	Scripts desarrollados	49
6.3.	Documentación y dependencias	50
6.4.	Datos originales utilizados	50
6.5.	Datos procesados generados	51
6.6.	Archivos destinados a Power BI	52
6.7.	Disponibilidad del código y los datos	53
7.	Conclusiones.....	54

7.1.	Conclusiones del proyecto	54
7.2.	Relación entre objetivos y resultados obtenidos	55
7.3.	Conclusiones personales y aprendizaje adquirido	56
8.	Limitaciones y prospectiva	58
8.1.	Limitaciones.....	58
8.2.	Trabajo futuro.....	60
8.3.	Prospectiva de la arquitectura	62
	Referencias bibliográficas.....	63
Anexo A.	Privacidad y protección de datos	66
Anexo B.	Estructura de carpetas del proyecto	69
Anexo C.	Script de ingesta y limpieza	69
Anexo D.	Script de modelado predictivo	75
Anexo E.	Script de exportación a Power BI	80
Anexo F.	Imágenes del Dashboard en Power BI.....	83
Anexo G.	Declaración de uso de herramientas de IA	87

Índice de figuras

Figura 1. Arquitectura general propuesta para el análisis predictivo y la visualización interactiva

Figura 2. Arquitectura aplicada al caso práctico de predicción de intensidad del tráfico urbano de Madrid.

Figura 3. Organización del repositorio donde se encuentra el proyecto

Figura 4. Estructura de carpetas del proyecto

Índice de tablas

- Tabla 1. Componentes principales de la arquitectura propuesta.
- Tabla 2. Tecnologías seleccionadas para la implementación.
- Tabla 3. Fuentes de datos utilizadas en el caso práctico.
- Tabla 4. Principales controles y transformaciones.
- Tabla 5. Estructura del dataset analítico final
- Tabla 6. Métricas de evaluación del modelo predictivo.
- Tabla 7. Variables más importantes según importancia.
- Tabla 8. Scripts desarrollados durante el proyecto.
- Tabla 9. Archivos procesados generados por la arquitectura.
- Tabla 10. Relación entre los objetivos específicos y los resultados obtenidos

1. Introducción

La generación y la disponibilidad de grandes volúmenes de datos han transformado la forma en que las organizaciones, administraciones públicas y otras entidades pueden estudiar fenómenos complejos y fundamentar sus decisiones. El interés de los datos masivos no se basa solamente en su cantidad, sino también en la posibilidad de combinar fuentes heterogéneas, aplicar técnicas de procesamiento y de análisis para obtener el conocimiento que no sería fácilmente accesible a través de procedimientos tradicionales. Es por eso por lo que las tecnologías de Big Data y la analítica avanzada han impulsado nuevas formas de investigación y explotación de la información, aunque su utilidad depende de la calidad de los datos que se tengan, de las decisiones metodológicas y del correcto uso de las herramientas en función al problema que se quiere analizar (Kitchin, 2014).

Sin embargo, la disponibilidad de los datos no garantiza por sí misma que puedan utilizarse de manera adecuada. Los conjuntos de información pueden encontrarse distribuidos en diferentes archivos o plataformas, presentar estructuras y formatos diversos, contener registros incompletos o inconsistentes y necesitar transformaciones específicas antes de su análisis. Por ello, es necesario organizar procesos que permitan obtener, depurar, estructurar y conectar toda esa información, manteniendo la trazabilidad de las operaciones realizadas. La integración de estas fases dentro de un flujo de trabajo coherente facilita la construcción de los denominados conjuntos analíticos y su posterior utilización en modelos predictivos y herramientas de visualización.

Este Trabajo de Fin de Máster, titulado Diseño e implementación de una arquitectura de datos masivos para el análisis predictivo y la visualización interactiva, se centra en esta necesidad mediante el desarrollo de una solución tecnológica modular que integra las principales fases del ciclo de tratamiento de los datos. La propuesta se ha basado en una implementación local con Python y pandas para la ingesta y la transformación, archivos estructurados como soporte de intercambio y almacenamiento, scikit-learn para el análisis predictivo y Power BI para la visualización interactiva. El diseño muestra una separación funcional entre los procesos, de manera que cada componente dispone de una responsabilidad que se define y genera resultados que pueden ser utilizados por las fases de después.

Para comprobar el funcionamiento de la solución he seleccionado un caso práctico de movilidad urbana que se centra en el tráfico de Madrid. El Ayuntamiento de Madrid pone a disposición pública datos históricos de sus puntos de medida, junto con información complementaria sobre su ubicación y características. Estos datos permiten estudiar la evolución temporal y espacial del tráfico a partir de mediciones registradas por la infraestructura municipal. La utilización de este conjunto proporciona un escenario real de trabajo con archivos de gran volumen, observaciones temporales y datos geográficos, necesarios para desarrollar y evaluar un pipeline de procesamiento y análisis.

El caso que propongo estudiar utiliza 24 archivos mensuales correspondientes al periodo comprendido entre enero de 2023 y diciembre de 2024, además de un archivo de metadatos geográficos. A partir de un volumen de 308.458.911 registros originales, he aplicado un proceso de selección, limpieza y agregación sobre 50 puntos de medida. La versión del pipeline genera un dataset de 833.086 observaciones horarias completas, que constituye la base del análisis descriptivo y del modelo predictivo. Estas cifras describen el alcance de los datos procesados y no por ello tiene que ser una representación exhaustiva de toda la red viaria de Madrid

La finalidad del trabajo no se limita a la construcción de un cuadro de mando. Incluye el diseño de una arquitectura por componentes, la implementación de procesos de ingesta y preparación, la generación de un conjunto analítico, el entrenamiento y la evaluación de un modelo de regresión y la preparación de las tablas necesarias para su representación visual. En el ámbito predictivo, he utilizado Random Forest Regressor, un método basado en la combinación de árboles de decisión que permite abordar problemas de regresión y analizar la importancia de las variables utilizadas (Breiman, 2001). El diseño lo he creado para estimar la intensidad de tráfico de una hora objetivo a partir de información disponible en horas anteriores, evitando el uso de variables que no estarían disponibles en el momento de predicción.

La evaluación final la he realizado mediante una separación temporal entre los datos de entrenamiento y los de prueba, comparando el Random Forest con modelos de referencia basados en constancia y en la intensidad de la misma hora del día anterior. Sobre las observaciones evaluables de 2024, con el modelo he obtenido un MAE de 52,48 vehículos/hora, un RMSE de 89,29 vehículos/hora y un coeficiente R^2 de 0,9565. Estos

resultados muestran un rendimiento favorable dentro del escenario experimental que he definido, aunque no constituyen una garantía de precisión para las horas sin datos suficientes ni demuestran un funcionamiento operativo en tiempo real. El análisis detallado de las métricas, la cobertura y los errores lo explico en el capítulo correspondiente a la contribución.

La arquitectura que he planteado sigue los criterios de modularidad y separación de responsabilidades para facilitar su mantenimiento y posible adaptación a otros contextos. No obstante, la validación que he realizado se limita al caso de tráfico seleccionado y a una ejecución local. Por tanto, la escalabilidad distribuida, la reutilización efectiva en otros dominios y el despliegue en producción se pueden considerar posibilidades de ampliación que requerirían pruebas adicionales.

1.1.Motivación

La motivación principal de este Trabajo Fin de Máster surge del interés por desarrollar una solución que permita recorrer de forma integrada el ciclo de tratamiento del dato, desde su obtención hasta su interpretación visual. En muchos proyectos, la atención se centra en una herramienta concreta o en el resultado final del análisis, mientras que las tareas de preparación, integración, validación y organización del flujo de trabajo resultan muy necesarias para garantizar que los resultados sean fiables y reproducibles. Lo que se dice sobre el Big Data destaca precisamente que la disponibilidad de nuevas capacidades analíticas debe acompañarse de una reflexión crítica sobre la calidad de la información y las condiciones en que se genera el conocimiento (Kitchin, 2014).

El ámbito de la movilidad urbana lo he elegido como escenario de aplicación por su relevancia y por las características de los datos que produce. Los sistemas de transporte generan información procedente de diferentes infraestructuras y tecnologías, cuyo análisis puede contribuir a comprender los patrones de circulación, estudiar la evolución de la demanda y apoyar la gestión de los servicios. Los sistemas inteligentes de transporte basados en datos integran distintas fuentes y técnicas analíticas para mejorar el conocimiento de los fenómenos de movilidad y facilitar la toma de decisiones (Zhang et al., 2011).

Desde el punto de vista académico, el proyecto permite aplicar de manera conjunta lo aprendido dentro del Máster Universitario en Análisis y Visualización de Datos Masivos. El

trabajo integra programación, tratamiento de grandes volúmenes de registros, preparación de datos, aprendizaje automático, evaluación de modelos y diseño de visualizaciones interactivas. La combinación de estas disciplinas ofrece una oportunidad para estudiar no solo el funcionamiento individual de cada herramienta, sino también las decisiones necesarias para conectarlas dentro de una solución software coherente.

La motivación técnica se refuerza por las dificultades encontradas durante el desarrollo. La interpretación de las unidades de intensidad, la agregación de mediciones a escala horaria, la existencia de huecos temporales y la disponibilidad de las variables predictivas han exigido revisar el procesamiento y el protocolo de evaluación. Estas cuestiones muestran que la calidad del resultado no depende únicamente del algoritmo utilizado, sino también de la correcta definición de la variable objetivo y del control de la información disponible en cada instante. Por ello, el trabajo incorpora una perspectiva de validación que contempla tanto la preparación de los datos como el rendimiento predictivo y el funcionamiento del conjunto de la solución.

Finalmente, la elección del tema corresponde al interés por desarrollar una contribución práctica de tipo software que pueda ser comprendida, ejecutada y ampliada posteriormente. La arquitectura propuesta pretende servir como base organizada para futuros desarrollos, sin afirmar que se haya demostrado su funcionamiento en cualquier dominio ni que sustituya a las plataformas distribuidas especializadas. Su valor se encuentra en la integración de componentes, la explicitación de sus responsabilidades y la posibilidad de evaluar de forma trazable los resultados obtenidos.

1.2. Planteamiento del trabajo

El problema abordado en este TFM consiste en diseñar e implementar una solución que integre de manera coherente las fases necesarias para transformar datos históricos de gran volumen en información analítica y predictiva accesible mediante un cuadro de mando interactivo. Aunque existen herramientas especializadas para la ingesta, el almacenamiento, el procesamiento, el aprendizaje automático y la visualización, su utilización conjunta requiere definir interfaces, estructuras de datos, criterios de calidad y procedimientos de ejecución que permitan mantener la consistencia entre las distintas etapas.

Para dar respuesta a esta necesidad, he desarrollado una arquitectura modular organizada en componentes funcionales. El flujo comienza con la adquisición de los archivos originales, continúa con su ingesta y transformación, genera un dataset analítico estructurado, incorpora un módulo de predicción y finaliza con la preparación de resultados para Power BI. La comunicación entre fases se realiza mediante archivos CSV y una estructura de directorios que diferencia los datos originales, los procesados y las exportaciones destinadas a visualización. Esta solución se ha implementado en un entorno local, utilizando herramientas accesibles y tecnologías adecuadas al alcance del proyecto.

La validación práctica la he realizado sobre el histórico de tráfico de Madrid de 2023 y 2024. El procesamiento permite seleccionar los puntos de medida, aplicar reglas de calidad, construir observaciones horarias y asociar información geográfica. Posteriormente, el módulo predictivo genera variables históricas por sensor, establece una división temporal de los datos y entrena un Random Forest Regressor. La evaluación compara sus resultados con baselines temporales mediante MAE, RMSE y R^2 , mientras que el dashboard facilita la exploración de los datos, los patrones de tráfico, la información geográfica y los resultados del modelo.

En el planteamiento se incluye también la necesidad de comprobar que las salidas de cada fase son coherentes y que el sistema puede reproducirse siguiendo un procedimiento definido. Por ello, se contemplan controles de integridad, revisión de la cobertura, comprobación de duplicados, evaluación de las predicciones y documentación de los archivos y dependencias. Las pruebas funcionales, de rendimiento y de usabilidad o aceptación que se incorporen a la versión final deberán presentarse con sus resultados reales, diferenciando las comprobaciones ya realizadas de aquellas que todavía estén pendientes.

El trabajo se enmarca principalmente en el Tipo 2 – Desarrollo de software, conforme a la clasificación establecida por UNIR. La aportación central es una solución tecnológica implementada y evaluada, no la formulación de una metodología nueva. La guía del título dice que este tipo de trabajo debe concretarse en una solución software funcional, con una demostración práctica y procedimientos de validación. En consecuencia, el desarrollo se orienta a documentar los requisitos, justificar las decisiones de diseño y aportar evidencias del funcionamiento de los componentes implementados.

Aunque el diseño se concibe con criterios de modularidad, su alcance experimental queda delimitado por el caso de estudio, los recursos locales utilizados y la selección de sensores. No se ha implementado una plataforma distribuida basada en Spark ni un sistema de almacenamiento PostgreSQL, y tampoco se ha desplegado un servicio de predicción en tiempo real. Estas tecnologías y capacidades podrán analizarse como alternativas o líneas futuras, pero no forman parte de los resultados ejecutados. Del mismo modo, la posibilidad de reutilizar los componentes en otros dominios constituye una propiedad de diseño que requeriría validación específica para poder considerarse demostrada.

1.3. Estructura del trabajo

La memoria se organiza en ocho capítulos principales, además de las referencias bibliográficas y los anexos. La estructura sigue la plantilla de UNIR y permite presentar de forma diferenciada los fundamentos del trabajo, la metodología, la contribución desarrollada y las conclusiones obtenidas.

El capítulo 1 introduce el problema, presenta la motivación que justifica la elección del tema, delimita el planteamiento del proyecto y describe la organización del documento.

El capítulo 2 desarrolla el contexto y el estado del arte. Se revisan los fundamentos de las arquitecturas de datos masivos, los enfoques de integración y procesamiento, las técnicas de predicción aplicadas a series temporales de tráfico y los principios de visualización interactiva. Asimismo, se analizan soluciones y trabajos relacionados con el fin de identificar sus aportaciones, limitaciones y la posición que ocupa la propuesta desarrollada.

El capítulo 3 presenta los objetivos concretos y la metodología. Se establece un objetivo general y un conjunto reducido de objetivos específicos, asociados a resultados y evidencias verificables. También se describe el proceso de desarrollo seguido, las fases del proyecto y los criterios utilizados para evaluar el cumplimiento de los objetivos.

El capítulo 4 aborda el marco normativo aplicable. Se examinan las condiciones de reutilización de los datos públicos, las consideraciones relacionadas con la protección de datos personales y las obligaciones que puedan afectar al tratamiento, almacenamiento y publicación de información. Se incluye una valoración específica de la naturaleza de los datos empleados y de los requisitos de privacidad pertinentes para el caso de estudio.

El capítulo 5 recoge el desarrollo específico de la contribución y constituye el núcleo técnico de la memoria. En él se describen los requisitos funcionales y no funcionales, la arquitectura implementada, las fuentes de datos, los procesos de ingesta y limpieza, la construcción del dataset analítico, el modelo predictivo y su protocolo de evaluación. También se documentan el diseño del dashboard, el modelo de datos de Power BI, las medidas e interacciones relevantes, los resultados obtenidos y las pruebas de validación del software. La exposición se apoya en tablas, diagramas, fragmentos de código y capturas que permiten relacionar las decisiones de implementación con sus evidencias.

El capítulo 6 está dedicado al código fuente y a los datos analizados. Se presenta la organización del repositorio, la estructura de directorios, las dependencias, las instrucciones de instalación y ejecución y la descripción de los principales archivos de entrada y salida. Asimismo, se documentan las condiciones de acceso a los datos originales y los elementos necesarios para reproducir el procesamiento y el análisis.

El capítulo 7 expone las conclusiones del trabajo. Se relacionan los resultados obtenidos con los objetivos planteados, se valora el grado de cumplimiento de cada uno y se sintetizan las aportaciones técnicas y analíticas de la solución desarrollada.

Por último, el capítulo 8 presenta las limitaciones y la prospectiva. Se analizan las restricciones asociadas a la selección y cobertura de sensores, la disponibilidad de datos, el alcance de la evaluación predictiva y la ejecución local. También se proponen posibles líneas de mejora, como la ampliación a otros conjuntos de datos, la automatización de la ingesta, la evaluación de tecnologías distribuidas y el despliegue de servicios de predicción, diferenciando estas posibilidades del trabajo efectivamente realizado.

2. Contexto y estado del arte

2.1.Contexto del problema

El desarrollo de las tecnologías digitales ha incrementado la cantidad y diversidad de información disponible para su análisis. Las organizaciones y administraciones públicas generan datos mediante aplicaciones, sistemas de gestión, plataformas web, dispositivos conectados y sensores, entre otras fuentes. En este contexto, el concepto de Big Data hace referencia a conjuntos de datos cuyo volumen, complejidad o características de generación pueden superar las capacidades de las herramientas tradicionales de gestión y procesamiento, haciendo necesario emplear enfoques y tecnologías adaptados a sus necesidades.

El Big Data suele describirse mediante cinco dimensiones: volumen, velocidad, variedad, veracidad y valor. El volumen se refiere a la cantidad de información; la velocidad, al ritmo de generación y procesamiento; la variedad, a la existencia de diferentes tipos y formatos; la veracidad, a la fiabilidad y calidad de los datos; y el valor, a la utilidad que puede obtenerse de su análisis. Estas dimensiones permiten comprender que un proyecto de datos masivos no se define únicamente por el tamaño de sus archivos, sino también por la complejidad de su tratamiento y por los objetivos analíticos que persigue.

Una arquitectura de datos establece la forma en que la información se recopila, transforma, almacena y distribuye para su utilización. Su diseño debe responder a los requisitos del proyecto y definir los componentes, las estructuras y los flujos necesarios para que los datos puedan utilizarse de manera coherente. La arquitectura no es, por tanto, una simple selección de herramientas, sino una organización de responsabilidades y procesos que facilita la integración y el mantenimiento del sistema.

En el ámbito de la movilidad urbana, la disponibilidad de información procedente de sensores, infraestructuras y sistemas de transporte ofrece oportunidades para estudiar el comportamiento de la circulación y apoyar la gestión de los servicios. Los sistemas inteligentes de transporte basados en datos combinan información de diferentes fuentes y técnicas analíticas para mejorar el conocimiento de las condiciones de movilidad, aunque su aplicación requiere tener en cuenta aspectos como la calidad de los datos, la integración y las condiciones de despliegue (Zhang et al., 2011).

Madrid constituye un escenario adecuado para estudiar estas necesidades. El Portal de Datos Abiertos del Ayuntamiento publica información histórica de los puntos de medida de tráfico desde julio de 2013 y añade periódicamente los datos de los meses anteriores. Además, dispone de conjuntos relacionados con la ubicación de los sensores y con información de tráfico actualizada casi en tiempo real. Esta disponibilidad permite diferenciar entre el análisis de históricos y las posibles aplicaciones operativas que requerirían una conexión continua con las fuentes.

El problema no queda resuelto únicamente con la publicación de los datos. Para obtener información analítica fiable es necesario conocer la estructura de los archivos, interpretar correctamente las variables, controlar los registros incompletos y organizar las transformaciones. En el caso desarrollado en este TFM, estas necesidades se concretan en el tratamiento de mediciones temporales de tráfico, su agregación a una granularidad horaria, la construcción de variables históricas y la preparación de resultados para un modelo predictivo y un cuadro de mando interactivo.

La contribución se sitúa en la integración de estas fases dentro de una solución modular. El proyecto no pretende demostrar que no existan herramientas capaces de realizar el ciclo completo del dato, sino diseñar e implementar una combinación concreta de componentes adecuada al alcance del caso de estudio y evaluar su funcionamiento. La movilidad urbana actúa como escenario de validación de la propuesta, mientras que la posible adaptación a otros dominios se considera una característica de diseño pendiente de comprobar experimentalmente.

2.2. Estado del arte

2.2.1. Arquitecturas Big Data para el análisis de datos masivos

Las arquitecturas Big Data surgen como respuesta a la necesidad de gestionar conjuntos de datos cuyo volumen, variedad o complejidad requieren enfoques de almacenamiento y procesamiento adaptados. Una arquitectura de datos masivos organiza los componentes que intervienen en la ingesta, almacenamiento, transformación, análisis y explotación de la información, estableciendo cómo se comunican y qué responsabilidad asume cada uno. Su diseño depende de los requisitos del sistema, de la naturaleza de las fuentes y de las

necesidades de rendimiento y disponibilidad, por lo que no existe una única arquitectura adecuada para todos los escenarios.

Entre los enfoques de almacenamiento más utilizados se encuentran los almacenes de datos, los data lakes y los lakehouses. Un almacén de datos o data warehouse integra información procedente de distintas fuentes en un repositorio optimizado para consultas y análisis. Habitualmente, los datos se limpian y estructuran mediante procesos ETL o ELT, lo que facilita su utilización en sistemas de inteligencia empresarial y en la elaboración de indicadores.

Por otra parte, un data lake permite almacenar grandes volúmenes de datos en su formato original, incluyendo información estructurada, semiestructurada y no estructurada. Este enfoque ofrece flexibilidad para conservar fuentes heterogéneas y decidir posteriormente qué transformaciones aplicar. Sin embargo, requiere mecanismos de catalogación, metadatos, calidad y gobierno para evitar que la información almacenada resulte difícil de localizar o utilizar. Los lakehouses combinan características de ambos enfoques, incorporando capacidades de gestión y análisis sobre un almacenamiento flexible.

Característica	Data warehouse	Data lake	Lakehouse
Finalidad principal	Consulta y análisis estructurado	Conservación flexible de datos diversos	Integración de almacenamiento y análisis
Tipo de datos	Principalmente estructurados	Estructurados, semiestructurados y no estructurados	Diferentes tipos y formatos
Tratamiento	Datos preparados para el análisis	Puede conservar datos originales	Combina datos originales y preparados
Ventaja principal	Coherencia y optimización analítica	Flexibilidad de almacenamiento	Integración de distintas cargas analíticas
Limitación o reto	Modelado y mantenimiento	Gobierno y organización de datos	Mayor complejidad de infraestructura

Otra decisión fundamental es el tipo de procesamiento. El procesamiento por lotes (batch processing) trabaja sobre conjuntos de datos disponibles en un momento determinado y resulta adecuado para históricos, análisis periódicos y tareas que no requieren una respuesta inmediata. El procesamiento en tiempo real o de flujos (stream processing) se orienta a tratar eventos conforme se reciben, por lo que exige mecanismos de ingesta continua y requisitos específicos de latencia. Ambos enfoques pueden coexistir dentro de una misma arquitectura, aunque su elección depende de las necesidades del proyecto.

Entre los patrones que combinan estas necesidades destaca la arquitectura Lambda, que utiliza una ruta de procesamiento por lotes y otra de baja latencia. Esta separación permite disponer de resultados históricos y recientes, pero puede incrementar la complejidad al mantener diferentes rutas de procesamiento. La arquitectura Kappa plantea una alternativa basada en un flujo unificado de eventos que puede reproducirse para recalcular resultados. Ambos patrones están orientados a escenarios con requisitos de procesamiento continuo o combinación de datos históricos y recientes.

En este TFM se ha adoptado una arquitectura modular de procesamiento por lotes, ejecutada en un entorno local. La solución separa las funciones de adquisición de datos, ingesta y limpieza, almacenamiento de resultados, preparación analítica, modelado predictivo y visualización. Estas funciones se implementan mediante scripts Python y bibliotecas especializadas, utilizando archivos CSV como mecanismo de intercambio entre las fases y Power BI como entorno de explotación visual. La arquitectura permite mantener diferenciadas las responsabilidades de cada componente y facilita la revisión de sus entradas y salidas.

Aunque el diseño se ha planteado con criterios de modularidad y posibilidad de ampliación, la implementación no constituye una arquitectura Lambda o Kappa, ni utiliza un motor distribuido como Apache Spark. Tampoco se ha implementado un data lake empresarial ni una base de datos PostgreSQL. La escalabilidad distribuida y la adaptación efectiva a otros dominios se consideran posibles líneas futuras que requerirían pruebas adicionales. Esta delimitación permite distinguir las características del diseño de las capacidades que han sido demostradas en este caso práctico.

2.2.2. Análisis predictivo aplicado a la movilidad urbana

La movilidad urbana constituye un ámbito adecuado para aplicar técnicas de análisis predictivo debido a la disponibilidad de datos temporales y geográficos procedentes de sensores, sistemas de transporte e infraestructuras. La predicción de tráfico a corto plazo busca estimar las condiciones futuras de circulación a partir de información histórica y, en determinados sistemas, de datos recientes. Su utilidad potencial se relaciona con la planificación, la gestión del tráfico y la anticipación de situaciones de congestión, aunque el rendimiento de los modelos depende de la calidad de las observaciones y de las características del entorno analizado.

Los enfoques utilizados para la predicción de tráfico incluyen modelos estadísticos de series temporales, métodos de aprendizaje automático supervisado y técnicas de aprendizaje profundo. Los modelos estadísticos permiten representar relaciones temporales mediante estructuras matemáticas definidas, mientras que los métodos de aprendizaje automático pueden capturar relaciones no lineales entre variables históricas, temporales y otras características disponibles. La elección de un método debe considerar la disponibilidad de datos, el horizonte de predicción, la complejidad computacional y la interpretación de los resultados, ya que no existe un algoritmo que sea superior en todos los escenarios.

Dentro de los métodos supervisados, Random Forest es un algoritmo basado en la combinación de múltiples árboles de decisión. En problemas de regresión, las predicciones de los árboles se agregan para obtener una estimación final. El método introduce aleatoriedad mediante la selección de muestras y variables, lo que permite reducir la dependencia de un único árbol y mejorar la capacidad de generalización en comparación con árboles individuales. Además, proporciona mecanismos para estudiar la importancia relativa de las variables utilizadas.

En este TFM se ha seleccionado Random Forest Regressor por su capacidad para trabajar con relaciones no lineales, combinar variables temporales e históricas y ofrecer un equilibrio adecuado entre complejidad, rendimiento e interpretabilidad para el alcance del proyecto. La elección no implica que sea el método óptimo para cualquier problema de tráfico, sino que constituye una alternativa adecuada para construir y evaluar un modelo predictivo dentro de la arquitectura desarrollada.

Un aspecto esencial en la predicción temporal es definir correctamente el horizonte de predicción y la información disponible en el momento en que se genera la estimación. La fuga de información (data leakage) se produce cuando el modelo utiliza durante su construcción información que no estaría disponible en una situación real de predicción. Este problema puede generar estimaciones excesivamente optimistas del rendimiento y dificultar la generalización a datos nuevos.

En la versión corregida del proyecto, el objetivo consiste en estimar la intensidad horaria de un sensor utilizando información histórica previa. Para ello, se han incorporado retardos de intensidad de una hora y de veinticuatro horas, medias móviles calculadas exclusivamente con observaciones anteriores y variables temporales conocidas. Las variables operativas, como ocupación, carga y velocidad, se utilizan con retardo cuando están disponibles, evitando emplear los valores contemporáneos de la hora que se pretende predecir. De este modo, la construcción de las características responde a un escenario de predicción de una hora de antelación, aunque su utilización operativa requeriría garantizar la disponibilidad efectiva de los datos anteriores en el instante de ejecución.

La evaluación de modelos de series temporales debe respetar el orden cronológico para evitar entrenar con información futura y evaluar sobre observaciones anteriores. A diferencia de una partición aleatoria convencional, una separación temporal permite representar de forma más realista el proceso de entrenamiento y posterior predicción. La documentación de scikit-learn recomienda utilizar estrategias de validación adaptadas a datos ordenados temporalmente cuando las particiones convencionales puedan introducir información futura.

El protocolo del TFM utiliza los primeros nueve meses de 2023 para el entrenamiento inicial, el último trimestre de ese año para la validación y el año 2024 como conjunto de prueba final. Tras la validación, el modelo se reentrena con los datos de 2023 y se evalúa sobre 2024. Además, se compara con dos modelos de referencia: la persistencia de la hora anterior y la intensidad de la misma hora del día anterior. Esta comparación permite determinar si el Random Forest aporta una mejora respecto a estrategias predictivas sencillas, en lugar de valorar su rendimiento únicamente mediante métricas aisladas.

Las métricas utilizadas son el error absoluto medio (MAE), la raíz del error cuadrático medio (RMSE) y el coeficiente de determinación (R^2). El MAE expresa el error absoluto medio en las mismas unidades que la variable objetivo, mientras que el RMSE penaliza en mayor medida los errores elevados. R^2 permite valorar la proporción de variabilidad explicada respecto a una referencia basada en la media, pero no debe interpretarse como un porcentaje de predicciones correctas. La evaluación debe considerar conjuntamente estas métricas y la cobertura de observaciones disponibles.

2.2.3. Visualización interactiva de datos de movilidad

La visualización interactiva constituye una fase relevante del análisis de datos porque permite representar información compleja mediante gráficos, mapas, tablas e indicadores que facilitan su exploración. En los sistemas de inteligencia empresarial, los informes interactivos permiten que el usuario seleccione periodos, categorías o elementos concretos y observe cómo cambian las visualizaciones en función de los filtros aplicados. Esta capacidad facilita el análisis desde diferentes perspectivas y permite pasar de una consulta general a un nivel de detalle más específico.

En el caso de la movilidad urbana, la dimensión temporal y geográfica de los datos hace especialmente útiles los gráficos de evolución, los perfiles horarios, los mapas y los rankings de puntos de medida. Los gráficos temporales permiten estudiar tendencias y patrones recurrentes, mientras que los mapas facilitan la localización espacial de los sensores y la comparación de sus indicadores. Las tablas de detalle complementan estas representaciones al permitir consultar valores concretos y detectar observaciones que requieren un análisis adicional.

La selección de visualizaciones debe responder a la naturaleza de los datos y al objetivo de cada análisis. Los gráficos de líneas son adecuados para representar tendencias temporales, los gráficos de barras permiten comparar categorías y los mapas resultan útiles cuando la ubicación espacial aporta información relevante. Asimismo, el uso de filtros, títulos claros, unidades y una organización visual coherente contribuye a que el informe pueda ser interpretado por usuarios con diferentes niveles de conocimiento técnico.

Para este TFM he utilizado Power BI Desktop, una aplicación que permite conectar fuentes de datos, transformarlas y construir informes visuales interactivos. La herramienta incluye

funcionalidades de modelado, creación de medidas DAX, filtros, segmentadores y diferentes tipos de visualizaciones. Su utilización resulta adecuada para integrar los resultados del procesamiento y del modelo predictivo dentro de un entorno de consulta común.

El informe que he desarrollado se organiza en ocho páginas: portada, visión general, calidad de datos, análisis temporal, análisis geográfico, modelo predictivo, análisis de errores y arquitectura y metodología. Esta estructura permite separar los distintos objetivos analíticos y facilitar la navegación entre ellos. Las visualizaciones se alimentan de tablas procesadas y exportadas por los scripts, de manera que Power BI actúa como capa de consulta y presentación de resultados, sin sustituir los procesos de limpieza y modelado realizados previamente.

Es importante diferenciar el concepto general de cuadro de mando del término técnico utilizado por Microsoft. En Power BI, un dashboard es un panel de una sola página disponible en el servicio Power BI, mientras que un informe puede contener varias páginas y permite una exploración más detallada. Por tanto, aunque en este TFM se utilice de forma general el término “cuadro de mando”, el archivo desarrollado en Power BI Desktop corresponde técnicamente a un informe interactivo multipágina.

2.2.4. Herramientas y soluciones existentes

Existen diferentes tecnologías que pueden utilizarse para implementar las fases de una arquitectura de datos. Su selección debe responder al problema concreto, a los recursos disponibles y a los requisitos de procesamiento, almacenamiento y visualización. En el presente trabajo se han considerado herramientas de procesamiento distribuido, bibliotecas de análisis de datos y plataformas de inteligencia empresarial, diferenciando las alternativas tecnológicas de los componentes que finalmente se han implementado.

Apache Spark es un motor de procesamiento de datos que permite ejecutar operaciones distribuidas sobre grandes conjuntos de información. Incluye funcionalidades para trabajar con datos estructurados, consultas SQL, procesamiento de flujos y aprendizaje automático mediante MLlib. Estas capacidades lo convierten en una alternativa relevante para arquitecturas que requieren procesamiento distribuido, aunque su utilización implica requisitos de infraestructura y configuración que deben justificarse en función de la escala del proyecto.

Python, junto con bibliotecas como pandas, ofrece un entorno flexible para la preparación y análisis de datos estructurados. pandas proporciona estructuras como DataFrame y operaciones de lectura, filtrado, agrupación y transformación, además de permitir procesar archivos por bloques cuando resulta necesario controlar el consumo de memoria. Estas características lo hacen adecuado para construir pipelines locales y realizar tareas de preparación de datos, aunque el procesamiento por bloques no equivale a una ejecución distribuida.

En el ámbito predictivo, scikit-learn proporciona algoritmos de aprendizaje automático supervisado y no supervisado, herramientas de preprocesamiento y funciones para la evaluación de modelos. Su biblioteca incluye RandomForestRegressor, que ha sido el algoritmo utilizado en el proyecto. La integración con pandas y NumPy facilita construir un flujo de preparación, entrenamiento, predicción y evaluación dentro del entorno Python.

En cuanto a las herramientas de visualización, Power BI y Tableau permiten crear informes interactivos, conectar diferentes fuentes y representar indicadores mediante gráficos, mapas y tablas. Ambas ofrecen funcionalidades de exploración y análisis visual, aunque difieren en sus entornos de desarrollo, modelos de datos, integración con otras plataformas y opciones de publicación. Para el presente TFM se ha seleccionado Power BI por su integración con el entorno de trabajo utilizado y por las posibilidades de modelado y creación de medidas DAX, sin que ello implique que Tableau sea una alternativa inferior en términos generales.

Herramienta	Función principal
Python y pandas	Ingesta, limpieza y transformación
scikit-learn	Entrenamiento y evaluación predictiva
Archivos CSV	Almacenamiento e intercambio de resultados
Power BI Desktop	Modelado y visualización interactivaImplementado

El análisis de estas soluciones muestra que existen tecnologías capaces de cubrir individualmente o de forma integrada diferentes fases del ciclo de vida del dato. Por ello, el problema abordado no debe formularse como una ausencia general de herramientas o arquitecturas completas. La aportación del TFM consiste en diseñar, implementar y evaluar

una combinación concreta de componentes que permita recorrer el proceso desde los datos históricos de tráfico hasta la predicción y su visualización, con un alcance delimitado y una organización que facilite su mantenimiento y posible ampliación.

La diferenciación respecto a otras soluciones se fundamenta en el enfoque integrado del caso práctico, en la trazabilidad del procesamiento, en la corrección del protocolo predictivo y en la incorporación de visualizaciones que permiten analizar tanto los datos como los errores del modelo. No se plantea como una arquitectura universal ni como una alternativa equivalente a plataformas distribuidas empresariales, sino como una solución software funcional y evaluable para el contexto definido.

2.3.Conclusiones

La revisión del estado del arte permite concluir que las arquitecturas de datos masivos proporcionan un marco para organizar el tratamiento de información procedente de fuentes heterogéneas, integrando procesos de ingesta, almacenamiento, transformación, análisis y explotación. La elección de una arquitectura debe responder a las características de los datos y a los requisitos del sistema, ya que no todos los escenarios requieren las mismas tecnologías ni una infraestructura distribuida. En este sentido, los enfoques por lotes, las arquitecturas orientadas a flujos y los distintos modelos de almacenamiento ofrecen alternativas que deben seleccionarse en función del contexto de aplicación.

El análisis predictivo aplicado a la movilidad urbana constituye un campo de interés por su capacidad para estimar condiciones futuras de tráfico y apoyar, potencialmente, la planificación y gestión de los servicios de transporte. La literatura recoge diferentes enfoques, desde modelos estadísticos de series temporales hasta algoritmos de aprendizaje automático y aprendizaje profundo. Sin embargo, la elección del modelo debe considerar el horizonte de predicción, la disponibilidad de información, la calidad de los datos y la comparación con métodos de referencia, en lugar de basarse únicamente en la complejidad del algoritmo.

Asimismo, la revisión pone de manifiesto la importancia de preparar correctamente los datos temporales y evitar la fuga de información durante el entrenamiento y la evaluación. La utilización de variables que no estarían disponibles en el momento de realizar una predicción puede producir estimaciones artificialmente optimistas del rendimiento. Por ello, resulta

necesario definir con precisión el horizonte de predicción, construir las variables a partir de información anterior y establecer una evaluación que respete el orden temporal.

La visualización interactiva complementa el procesamiento y el análisis predictivo al facilitar la exploración de resultados desde diferentes perspectivas. En el ámbito del tráfico, los gráficos temporales, mapas, indicadores y filtros permiten estudiar patrones horarios, comparar puntos de medida y examinar el comportamiento del modelo. La organización de estos elementos en un informe multipágina contribuye a presentar la información de forma estructurada y accesible, siempre que se utilicen magnitudes correctamente definidas y se indiquen las limitaciones de los datos representados.

Finalmente, el análisis de las herramientas y enfoques existentes muestra que existen plataformas capaces de integrar distintas fases del ciclo de vida del dato, así como tecnologías especializadas para cada una de ellas. Por tanto, la aportación diferencial de este TFM no se fundamenta en afirmar que no existan soluciones completas, sino en el diseño, implementación y evaluación de una combinación concreta de componentes adaptada a un caso de estudio real. La propuesta integra el procesamiento de históricos de tráfico de Madrid, la construcción de un dataset horario, un modelo predictivo evaluado temporalmente y un informe interactivo en Power BI.

La arquitectura desarrollada se concibe con criterios de modularidad y separación de responsabilidades, pero su validación se limita al escenario experimental definido y a una ejecución local. La posible ampliación hacia tecnologías distribuidas, automatización continua o aplicación a otros dominios se plantea como trabajo futuro. De este modo, la revisión bibliográfica justifica las decisiones técnicas adoptadas y establece el marco necesario para presentar los objetivos, la metodología y la contribución desarrollada en los capítulos siguientes.

3. Objetivos concretos y metodología de trabajo

Este capítulo presenta los objetivos del Trabajo Fin de Máster y describe el procedimiento seguido para desarrollar y evaluar la solución propuesta. A partir del contexto y del estado del arte expuestos en el capítulo anterior, la contribución se concreta en el diseño e implementación de una arquitectura modular orientada al procesamiento de datos históricos, el análisis predictivo y la visualización interactiva.

El trabajo se enmarca en la modalidad Tipo 2 – Desarrollo de software, ya que su aportación central consiste en una herramienta informática desarrollada específicamente para resolver un problema concreto. La guía de UNIR establece que esta modalidad debe incorporar una solución tecnológica funcional, una demostración práctica y una validación que contemple la usabilidad y/o aceptación por parte de expertos. Por ello, el proyecto no se limita al entrenamiento de un modelo ni al diseño de un informe visual, sino que aborda la integración de sus componentes y la comprobación de su funcionamiento.

3.1. Objetivo general

El objetivo general de este proyecto es diseñar, implementar y evaluar una arquitectura modular de datos orientada al análisis predictivo y la visualización interactiva, integrando procesos de ingesta, almacenamiento, transformación, modelado y presentación de resultados, mediante su aplicación a un caso práctico basado en datos abiertos de tráfico de Madrid correspondientes al periodo 2023–2024.

La finalidad es desarrollar una solución software que permita transformar registros históricos en conjuntos analíticos estructurados, generar predicciones horarias a partir de información disponible previamente y presentar los resultados mediante un informe interactivo en Power BI. La arquitectura se concibe como una solución organizada por componentes, de manera que sus procesos puedan mantenerse, revisarse y eventualmente adaptarse a otros contextos, aunque la validación experimental de este trabajo se limita al caso de estudio seleccionado.

El cumplimiento del objetivo general se valorará mediante evidencias del funcionamiento del pipeline, la calidad y cobertura de los datos obtenidos, el rendimiento predictivo frente a modelos de referencia, la funcionalidad del informe y la documentación necesaria para

reproducir la solución. No se pretende demostrar escalabilidad distribuida, funcionamiento en tiempo real ni reutilización efectiva en otros dominios, ya que estas capacidades quedan fuera del alcance implementado.

3.2. Objetivos específicos

Para alcanzar el objetivo general se establecen los siguientes objetivos específicos:

OE1. Definir los requisitos y diseñar la arquitectura de la solución. Identificar los requisitos funcionales y no funcionales del sistema, delimitar su alcance y diseñar una arquitectura modular que establezca las responsabilidades, entradas y salidas de los componentes de ingesta, procesamiento, análisis predictivo y visualización.

OE2. Obtener y preparar un conjunto de datos de tráfico adecuado para el análisis. Seleccionar y documentar las fuentes abiertas de Madrid, implementar los procesos de ingesta y limpieza, controlar la calidad y completitud de los registros y construir un dataset horario estructurado que integre información temporal, operativa y geográfica.

OE3. Desarrollar y evaluar un modelo de predicción horaria. Construir variables históricas que eviten la utilización de información futura, entrenar un modelo Random Forest Regressor y evaluar su rendimiento mediante una separación temporal y una comparación con modelos de referencia, utilizando métricas de regresión y análisis de errores.

OE4. Diseñar e implementar un informe interactivo para la exploración de los datos y resultados. Construir en Power BI un modelo de datos y un conjunto de páginas que permitan analizar la calidad de la información, los patrones temporales y geográficos, las predicciones y los errores, incorporando indicadores, filtros y mecanismos de navegación.

OE5. Verificar el funcionamiento y documentar la reproducibilidad de la solución. Definir y ejecutar pruebas funcionales, comprobar la coherencia de las salidas, evaluar aspectos de rendimiento y realizar una validación de usabilidad o aceptación. Documentar el código, las dependencias, los datos y las instrucciones necesarias para reproducir el flujo de trabajo.

OE6. Analizar los resultados, las limitaciones y las posibilidades de ampliación. Relacionar las evidencias obtenidas con los objetivos del proyecto, identificar las restricciones de los datos y

de la implementación local y valorar posibles mejoras, como la incorporación de nuevas fuentes, tecnologías distribuidas o mecanismos de automatización.

Objetivo	Evidencia principal	Criterio de comprobación
OE1	Requisitos y diagrama de arquitectura	Componentes, interfaces y alcance definidos
OE2	Dataset limpio e informes de calidad	Reglas aplicadas, duplicados, completitud y cobertura documentados
OE3	Modelo, predicciones y métricas	Evaluación temporal y comparación con baselines
OE4	Archivo PBIX y capturas	Páginas, filtros, indicadores e interacciones funcionales
OE5	Repositorio, instrucciones y pruebas	Ejecución reproducible y resultados de validación documentados
OE6	Conclusiones y limitaciones	Correspondencia entre objetivos, resultados y mejoras propuestas

Los objetivos se han formulado de manera que puedan evaluarse mediante entregables y resultados verificables. La tabla anterior servirá como referencia para el capítulo de conclusiones, donde se indicará el grado de cumplimiento de cada objetivo sin considerar alcanzadas aquellas capacidades para las que no se disponga de evidencias.

3.3. Metodología del trabajo

3.3.1. Enfoque metodológico

La metodología del TFM toma como referencia CRISP-DM (Cross-Industry Standard Process for Data Mining), un modelo de proceso utilizado para organizar proyectos de minería de datos. CRISP-DM contempla seis fases: comprensión del negocio, comprensión de los datos,

preparación de los datos, modelado, evaluación y despliegue. Su estructura no es estrictamente lineal, ya que permite regresar a fases anteriores cuando los resultados o los problemas detectados requieren revisar decisiones previas.

En este trabajo, CRISP-DM se utiliza como marco de referencia y se adapta a un proyecto académico de desarrollo software. La comprensión del negocio se interpreta como la definición del problema y de los requisitos del sistema; la preparación y el modelado se aplican al caso de tráfico; y la fase de despliegue se concreta en la entrega de un prototipo local ejecutable, su documentación y la preparación del informe interactivo. No se ha realizado un despliegue operativo en una infraestructura de producción.

El desarrollo se ha realizado de forma iterativa, revisando los componentes cuando las comprobaciones han puesto de manifiesto problemas de calidad o de validez metodológica. En particular, la revisión de la agregación horaria y de la disponibilidad temporal de las variables predictivas condujo a modificar el pipeline y a repetir el entrenamiento y la evaluación. Esta forma de trabajo es coherente con el carácter iterativo de CRISP-DM, en el que las fases pueden retroalimentarse en función de los resultados obtenidos.

Para la organización de las tareas he seguido un enfoque incremental, priorizando la finalización y revisión de cada componente antes de avanzar al siguiente. Si se dispone de evidencias de un tablero Kanban utilizado durante el proyecto, este podrá documentarse como herramienta de apoyo a la gestión de tareas. En caso contrario, no se considerará una metodología formal implantada ni se afirmará que se hayan medido indicadores propios de Kanban.

3.3.2. Fases del desarrollo

Fase 1. Definición del problema, alcance y requisitos

Se delimitó la contribución como una solución software modular y se seleccionó el tráfico de Madrid como escenario de validación. Se identificaron las funciones que debía proporcionar el sistema, los recursos disponibles y las restricciones asociadas a una implementación local. Asimismo, se establecieron los criterios que permitirían comprobar el funcionamiento de la ingesta, el procesamiento, la predicción y la visualización.

Esta fase incluye la definición de requisitos funcionales y no funcionales, que se desarrollarán en el capítulo 5. Los primeros describen las funciones que debe realizar el sistema, mientras que los segundos establecen características y condiciones de calidad, como rendimiento, mantenibilidad, usabilidad y reproducibilidad. El modelo de calidad ISO/IEC 25010 constituye una referencia para especificar y evaluar atributos de calidad del producto software (ISO/IEC, 2023).

Fase 2. Comprensión y selección de los datos

Se analizaron los conjuntos de datos abiertos de tráfico del Ayuntamiento de Madrid y la información geográfica de sus puntos de medida. Se seleccionó el periodo comprendido entre enero de 2023 y diciembre de 2024 y se revisaron las variables disponibles, su estructura, los formatos y los posibles problemas de calidad.

La selección de los 50 puntos de medida se realizó conforme al criterio implementado en el pipeline, que se documentará junto con sus limitaciones. No se considera que esta selección sea representativa de toda la red de Madrid, ya que no se ha realizado un procedimiento de muestreo espacial diseñado para garantizar dicha representatividad.

Fase 3. Diseño e implementación de la arquitectura

Se diseñó una arquitectura local organizada en componentes de ingesta, almacenamiento de resultados, preparación analítica, modelado predictivo y visualización. La implementación utiliza Python, pandas, scikit-learn, archivos CSV y Power BI. Cada fase genera salidas que pueden ser utilizadas por los componentes posteriores, lo que permite separar responsabilidades y revisar de forma independiente los resultados del procesamiento.

La solución se ha implementado mediante scripts diferenciados y una estructura de directorios que conserva los datos originales y los resultados procesados. Apache Spark, PostgreSQL y las plataformas cloud se han considerado alternativas tecnológicas o posibles ampliaciones, pero no forman parte de la implementación ejecutada.

Fase 4. Ingesta, limpieza y preparación del conjunto analítico

Se desarrollaron procesos para leer los archivos mensuales, seleccionar los sensores, comprobar la validez de los registros y construir observaciones horarias. La revisión de la versión inicial permitió corregir la agregación de la intensidad, evitando sumar mediciones

que representan tasas, y establecer un criterio de completitud basado en la presencia de las cuatro observaciones previstas para cada hora.

El procesamiento incorpora controles de duplicados, fechas, valores inválidos y cobertura temporal. Los resultados se almacenan en archivos estructurados y se generan informes de calidad que permiten conocer el número de registros procesados y las observaciones excluidas. Las reglas concretas y sus consecuencias sobre el conjunto final se explicarán en el capítulo 5.

Fase 5. Análisis exploratorio y modelado predictivo

A partir del dataset horario se realizó un análisis exploratorio de los patrones temporales y geográficos del tráfico. Posteriormente, se construyeron variables históricas por sensor para estimar la intensidad de la hora objetivo. El modelo utiliza retardos y medias móviles calculadas con observaciones anteriores, evitando incorporar información contemporánea que no estaría disponible en el momento de predicción.

El entrenamiento y la evaluación se organizaron cronológicamente: entrenamiento inicial entre enero y septiembre de 2023, validación entre octubre y diciembre de 2023 y prueba final sobre 2024, tras reentrenar el modelo con los datos de 2023. Se utilizaron como referencias la persistencia de la hora anterior y el valor correspondiente a la misma hora del día anterior. Las métricas seleccionadas fueron MAE, RMSE y R^2 , complementadas con un análisis de errores por periodo y sensor.

La separación temporal responde a la necesidad de evitar que el modelo se evalúe con información futura utilizada durante el entrenamiento. En los problemas de series temporales, las particiones deben respetar el orden de las observaciones cuando una división aleatoria pueda introducir información no disponible en el escenario de predicción.

Fase 6. Visualización interactiva

Los conjuntos de datos procesados y los resultados predictivos se prepararon para su importación en Power BI mediante archivos CSV. Se construyó un informe interactivo multipágina que integra indicadores, gráficos temporales, mapas, rankings, tablas y filtros.

El informe permite explorar distintas dimensiones del caso de estudio, incluyendo calidad de datos, análisis temporal, análisis geográfico, resultados predictivos y análisis de errores. Las medidas y relaciones se diseñaron para facilitar una interpretación coherente de las magnitudes, diferenciando los indicadores globales de las métricas que deben recalcularse en función de los filtros aplicados.

Fase 7. Evaluación, documentación y entrega

La evaluación se organiza en tres dimensiones complementarias. La dimensión técnica comprueba la ejecución de los procesos, la coherencia de las entradas y salidas y el cumplimiento de los requisitos funcionales. La dimensión analítica valora el rendimiento del modelo mediante métricas y baselines sobre el conjunto de prueba. La dimensión de uso examina la claridad, navegación y utilidad del informe mediante pruebas de usabilidad o aceptación.

Además, se contempla la medición del tiempo de ejecución y del consumo de recursos en tareas representativas, así como la documentación del entorno y de las dependencias para facilitar la reproducibilidad. Los resultados de estas pruebas deberán incorporarse al capítulo 5 una vez ejecutadas, especificando el procedimiento, el entorno, los criterios de aceptación y las incidencias detectadas. No se considerará demostrada una capacidad de escalabilidad o usabilidad únicamente por el hecho de haber diseñado el sistema con ese propósito.

La entrega incluirá el código fuente, los archivos de configuración y dependencias, las instrucciones de ejecución, la descripción de los conjuntos de datos y el archivo de Power BI. Este conjunto de elementos permitirá realizar una demostración práctica del prototipo y comprobar las funcionalidades desarrolladas, de acuerdo con los requisitos de la modalidad de desarrollo software de la universidad.

3.3.3. Trazabilidad y control de la evaluación

Para mantener la coherencia entre la planificación y los resultados, cada objetivo específico se relacionará con uno o varios apartados del desarrollo y con evidencias de cumplimiento. Los resultados de las pruebas se documentarán indicando el requisito evaluado, el procedimiento aplicado, el resultado obtenido y su valoración. Este enfoque permitirá distinguir entre funcionalidades implementadas, funcionalidades verificadas y mejoras pendientes.

La metodología adoptada permite presentar el proyecto como un desarrollo software organizado y evaluable, manteniendo una separación clara entre el diseño de la arquitectura, su implementación en el caso de tráfico y las posibles ampliaciones futuras. Los resultados obtenidos se utilizarán para valorar el cumplimiento de los objetivos en las conclusiones, evitando atribuir al sistema capacidades que no hayan sido comprobadas.

3.4. Uso de herramientas de inteligencia artificial

Durante el desarrollo de este Trabajo Fin de Máster se ha utilizado la herramienta de inteligencia artificial generativa ChatGPT (OpenAI) como apoyo en tareas concretas de elaboración y revisión del trabajo. En particular, se ha empleado para ordenar y estructurar el contenido de la memoria por apartados, revisar y comentar scripts de Python utilizados en el proyecto, y apoyar la elaboración de algunas imágenes o esquemas explicativos relacionados con la organización del trabajo y la arquitectura desarrollada.

La herramienta también se ha utilizado como apoyo en la revisión de la redacción y en la detección de incoherencias entre diferentes versiones del documento. No obstante, las decisiones metodológicas, la implementación técnica, la ejecución del código, la comprobación de resultados, la selección de fuentes y la redacción final del trabajo han sido revisadas y validadas por la autora.

Las respuestas generadas por la herramienta no se han incorporado como fuente académica sin revisión previa. Cuando ha sido necesario incluir contenido teórico, normativo o técnico, este se ha contrastado con bibliografía, documentación oficial o fuentes especializadas. El detalle del uso de la herramienta se recoge en el Anexo G.

4. Marco normativo

Este Trabajo Fin de Máster se basa en el diseño e implementación de una arquitectura de datos orientada al análisis predictivo y a la visualización interactiva, validada mediante un caso práctico de movilidad urbana. El marco normativo aplicable se centra en tres ámbitos: la protección de datos personales, la reutilización de información del sector público y la normativa relacionada con movilidad. A estos elementos se añaden consideraciones éticas y de seguridad derivadas del uso de datos abiertos y de la posible combinación de diferentes fuentes.

4.1. Protección de datos personales

En materia de protección de datos, las normas de referencia son el Reglamento (UE) 2016/679, Reglamento General de Protección de Datos (RGPD), y la Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales. Esta última adapta y complementa en España el marco europeo de protección de datos personales.

El RGPD se aplica al tratamiento de datos personales, entendidos como información relativa a una persona física identificada o identificable. Por tanto, la cuestión clave no es únicamente que los datos sean públicos o estén agregados, sino si permiten identificar directa o indirectamente a una persona. Los datos verdaderamente anónimos, es decir, aquellos respecto de los cuales la persona ya no puede ser identificada razonablemente, quedan fuera del ámbito material del RGPD; en cambio, los datos seudonimizados continúan siendo datos personales si existe la posibilidad de volver a asociarlos a una persona. Esta distinción es especialmente relevante cuando se trabaja con información de localización o movilidad.

En este TFM, los conjuntos utilizados corresponden a mediciones de tráfico registradas por puntos de medida de la red viaria de Madrid. Las variables empleadas describen características del tráfico, como intensidad, ocupación, carga, velocidad y localización del sensor, y no contienen nombres, documentos de identidad, matrículas de vehículos, identificadores personales ni trayectorias individuales. Por ello, con la información utilizada en la implementación, no se realiza un tratamiento directo de datos personales identificables.

Debe evitarse, no obstante, concluir de forma genérica que cualquier dato agregado queda automáticamente fuera del RGPD. La aplicabilidad de la normativa depende de si la información permite identificar a una persona teniendo en cuenta los medios razonablemente disponibles. En este proyecto, la evaluación se basa en la naturaleza concreta de los datos publicados y en el hecho de que las observaciones corresponden a condiciones agregadas del tráfico en puntos de medida, no a individuos concretos.

Aunque el conjunto de datos analizado no incorpora información personal identificable, se adoptan principios compatibles con una gestión responsable de la información. Entre ellos destacan la minimización de datos, la limitación de la finalidad y la documentación de las transformaciones aplicadas. Estas medidas permiten reducir riesgos innecesarios y mantener la trazabilidad del tratamiento realizado.

4.2. Reutilización de información del sector público

La utilización de datos publicados por administraciones públicas está relacionada con la Ley 37/2007, de 16 de noviembre, sobre reutilización de la información del sector público. Esta norma regula el régimen jurídico aplicable a la reutilización de documentos elaborados o custodiados por los organismos incluidos en su ámbito de aplicación, así como determinadas condiciones sobre datos públicos reutilizables.

La reutilización se refiere al uso de documentos del sector público por personas físicas o jurídicas para fines distintos de la actividad administrativa para la que fueron inicialmente producidos. La Ley 37/2007 permite la reutilización siempre que se respeten las condiciones establecidas por el organismo que pone la información a disposición, incluyendo, cuando proceda, obligaciones relativas a la atribución de la fuente, la no alteración del sentido de la información o la conservación de determinadas referencias.

En el caso concreto del Portal de Datos Abiertos del Ayuntamiento de Madrid, las condiciones generales permiten reutilizar los documentos tanto para fines comerciales como no comerciales. El portal utiliza, con carácter general, la licencia Creative Commons Attribution 4.0 International (CC BY 4.0), además de condiciones específicas para determinados conjuntos de datos.

Por tanto, en este TFM la reutilización de los datos tiene finalidad académica y se respetarán las condiciones de atribución y uso indicadas por el Ayuntamiento. En la memoria deberá identificarse el organismo de procedencia, el nombre del conjunto de datos, el periodo utilizado y, cuando resulte relevante, la fecha de consulta o descarga. Asimismo, cualquier transformación realizada se presentará como elaboración propia a partir de la fuente original, evitando que los resultados procesados puedan confundirse con datos oficiales publicados directamente por el Ayuntamiento.

4.3. Normativa relacionada con movilidad

El caso de estudio se sitúa en el ámbito de la movilidad urbana, por lo que resulta pertinente considerar la Ley 9/2025, de 3 de diciembre, de Movilidad Sostenible. Esta norma establece un marco general para la movilidad en España y pone énfasis en aspectos como la sostenibilidad, la seguridad, la accesibilidad, la digitalización y una mejor planificación de los sistemas de transporte.

La Ley de Movilidad Sostenible reconoce la relevancia de la información y la digitalización para mejorar el conocimiento del sistema de movilidad y apoyar su planificación. Este marco resulta coherente con el tipo de análisis realizado en el TFM, ya que la explotación de datos históricos permite estudiar patrones temporales y espaciales de tráfico y desarrollar herramientas de apoyo analítico.

No obstante, el alcance del trabajo es exclusivamente académico y experimental. La solución desarrollada no controla semáforos, no modifica rutas, no interviene sobre infraestructuras reales y no genera decisiones automatizadas con efectos jurídicos o materiales sobre ciudadanos. Tampoco se ha desplegado como servicio operativo de predicción en tiempo real. Por ello, la normativa de movilidad actúa principalmente como marco contextual del caso de estudio y no como regulación específica de un sistema crítico implantado.

4.4. Consideraciones éticas y de seguridad

El carácter abierto de los datos no elimina la necesidad de evaluar los posibles riesgos asociados a su tratamiento. La combinación de diferentes fuentes puede incrementar la

información disponible y, en determinados escenarios, generar riesgos de reidentificación que no eran evidentes al analizar cada conjunto de forma independiente. Por esta razón, la utilización de información pública debe realizarse respetando tanto sus condiciones de reutilización como los principios de privacidad y seguridad aplicables.

En el proyecto se aplican las siguientes medidas:

- Utilizar únicamente variables necesarias para los objetivos analíticos definidos.
- No incorporar identificadores personales o datos que permitan reconstruir trayectorias individuales.
- No intentar identificar personas a partir de información anonimizada o agregada.
- Mantener separadas las fuentes originales y los datos procesados para conservar la trazabilidad.
- Documentar las operaciones de limpieza, agregación y transformación.
- Citar correctamente el origen de todos los conjuntos de datos utilizados.
- Evitar publicar resultados que puedan presentarse erróneamente como información oficial del Ayuntamiento.
- Conservar únicamente la información necesaria para la ejecución y reproducibilidad del proyecto.

Estas medidas se corresponden con un enfoque preventivo adecuado para el alcance de la solución. En particular, la arquitectura trabaja con información agregada por punto de medida y por periodo temporal, sin procesar identificadores de vehículos o personas.

4.5. Aplicación al TFM

La aplicación del marco normativo al proyecto puede resumirse en tres aspectos. En primer lugar, los datos analizados no incluyen información personal identificable en la forma en que han sido utilizados, por lo que el riesgo asociado a la protección de datos personales es reducido. Esta valoración se limita a los archivos concretos empleados en el TFM y no debe generalizarse a cualquier conjunto de datos de movilidad.

En segundo lugar, los datos proceden de una fuente pública sometida a condiciones de reutilización. El uso académico realizado es compatible con las condiciones generales del

Portal de Datos Abiertos del Ayuntamiento de Madrid, siempre que se mantenga la atribución de la fuente y se respeten las condiciones específicas que puedan acompañar al conjunto de datos.

Finalmente, el sistema desarrollado es una herramienta de análisis y visualización, no un sistema de control de movilidad ni un servicio automatizado de toma de decisiones. La arquitectura se ejecuta localmente sobre datos históricos y los resultados predictivos se utilizan con finalidad académica. En consecuencia, el marco normativo se aplica fundamentalmente como referencia para el tratamiento responsable de los datos, la correcta reutilización de información pública y la contextualización del caso de movilidad urbana.

En conjunto, estas condiciones permiten desarrollar el TFM dentro de un marco de uso responsable de la información, manteniendo una distinción clara entre datos públicos, datos personales y resultados analíticos derivados.

5. Desarrollo específico de la contribución

Este capítulo constituye el núcleo técnico del Trabajo Fin de Máster y describe la solución desarrollada para integrar el procesamiento de datos, el análisis predictivo y la visualización interactiva dentro de un mismo flujo de trabajo. La contribución se concreta en una arquitectura modular implementada en entorno local y validada mediante un caso práctico de tráfico urbano de Madrid.

La solución desarrollada no se limita a una representación conceptual. Incluye la adquisición y organización de datos históricos, la implementación de procesos de ingesta y limpieza mediante Python, la generación de un conjunto analítico horario, la construcción y evaluación de un modelo predictivo y la preparación de los resultados para su explotación en Power BI. De este modo, el trabajo se corresponde con la modalidad de desarrollo de software, cuya aportación principal debe materializarse en una solución tecnológica funcional y evaluable. La guía de UNIR establece que, para este tipo de trabajo, la herramienta desarrollada constituye la contribución central y debe poder demostrarse de forma práctica.

La arquitectura se ha diseñado con criterios de modularidad, trazabilidad y separación de responsabilidades. No obstante, la validación realizada corresponde al caso de tráfico seleccionado y a una ejecución local, por lo que la escalabilidad distribuida y la reutilización efectiva en otros dominios no se consideran capacidades demostradas, sino posibilidades de evolución futura.

5.1. Descripción general de la arquitectura propuesta

La arquitectura implementada se organiza en seis bloques funcionales: adquisición de datos, ingesta, procesamiento y limpieza, construcción del dataset analítico, modelado predictivo y visualización interactiva. Cada bloque recibe unas entradas definidas, ejecuta un conjunto de operaciones y genera salidas que pueden ser utilizadas por el siguiente componente.

La arquitectura desarrollada se ejecuta de forma secuencial y por lotes sobre archivos históricos. Por tanto, no constituye una arquitectura de procesamiento en tiempo real ni un sistema distribuido. Los archivos intermedios permiten conservar la trazabilidad entre los datos originales, los datos procesados, las predicciones y las tablas destinadas a Power BI.

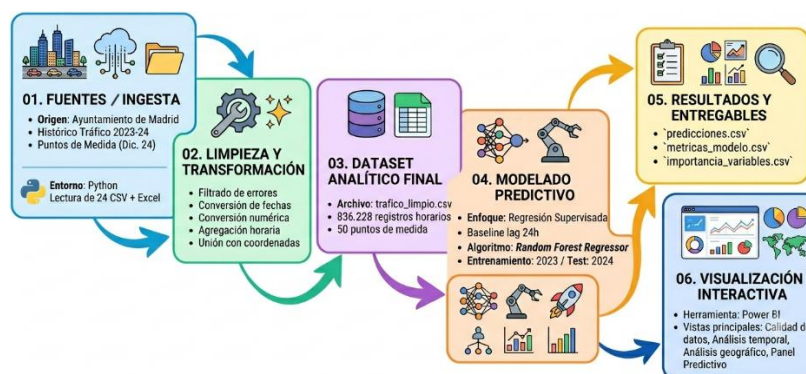


Figura 1. Arquitectura general propuesta para el análisis predictivo y la visualización interactiva. Elaboración IA

Capa	Función	Tecnologías
Adquisición de datos	Obtención de datos históricos y metadatos geográficos.	Portal de Datos abiertos del Ayuntamiento de Madrid
Ingesta	Lectura y selección de registros.	Python y pandas
Procesamiento y limpieza	Validación, tipado, filtrado y agregación horaria.	Python y pandas
Dataset analítico	Construcción de variables temporales, históricas y operativas.	CSV estructurados
Modelado predictivo	Entrenamiento, predicción y evaluación.	Scikit-learn
Visualización	Exploración de datos, métricas y resultados	Power BI Desktop

Tabla 1. Componentes principales de la arquitectura propuesta. Elaboración propia.

La arquitectura se basa en componentes accesibles y de uso general. Esto facilita la reproducción del entorno académico y permite inspeccionar las salidas de cada etapa. Sin embargo, esta elección implica que el sistema está limitado por los recursos del equipo local y no debe equipararse a una arquitectura distribuida empresarial.

5.2. Instanciación de la arquitectura en el caso de la movilidad urbana

Para validar la arquitectura se ha utilizado el tráfico urbano de Madrid como caso práctico. El Portal de Datos Abiertos del Ayuntamiento publica un conjunto histórico de datos de los puntos de medida de tráfico, actualizado mensualmente, junto con información complementaria sobre la ubicación de dichos puntos. El histórico está disponible desde julio de 2013 y dispone de documentación asociada sobre la estructura de los ficheros.

El caso de estudio se ha delimitado al periodo comprendido entre enero de 2023 y diciembre de 2024. Se han procesado 24 archivos mensuales y se ha utilizado información geográfica correspondiente a los puntos de medida para relacionar cada identificador con variables espaciales como distrito, longitud y latitud.

El problema predictivo se formula como la estimación de la intensidad de tráfico de una hora concreta para cada punto seleccionado, utilizando información disponible en instantes anteriores. La unidad analítica final es, por tanto, la pareja punto de medida hora.

La elección de una granularidad horaria responde a dos motivos. En primer lugar, permite representar de forma clara patrones diarios y semanales. En segundo lugar, reduce el número de observaciones respecto a la granularidad original y simplifica la construcción del modelo predictivo y del informe visual. La agregación, sin embargo, debe respetar la naturaleza de la variable medida; por ello, la versión corregida utiliza la media de las observaciones de intensidad de cada hora y exige que estén presentes las cuatro mediciones esperadas.

Este punto es importante porque la versión inicial sumaba las cuatro observaciones de cada hora, lo que multiplicaba artificialmente la escala de la variable objetivo. La corrección del procedimiento de agregación constituye una de las modificaciones principales introducidas respecto a la entrega anterior.

La validación se ha realizado sobre una selección de 50 puntos de medida. Esta selección permite trabajar con un volumen elevado y manejable en un entorno local, pero no se considera una muestra espacialmente representativa de toda la red viaria de Madrid, ya que no se diseñó un muestreo específico por distrito, tipo de vía o intensidad de circulación.



Figura 2. Arquitectura aplicada al caso práctico de predicción de intensidad del tráfico urbano de Madrid. Elaboración IA.

5.3. Tecnologías seleccionadas y justificación

La implementación se ha realizado utilizando Python como lenguaje principal. Su ecosistema permite integrar en un mismo entorno tareas de lectura, transformación, análisis y aprendizaje automático. En este proyecto, Python se utiliza para automatizar el pipeline y ejecutar los diferentes scripts que forman la solución.

La biblioteca *pandas* se emplea para el tratamiento tabular de los datos, incluyendo lectura de ficheros, filtrado, conversión de tipos, agrupaciones y exportación de resultados. El procesamiento inicial utiliza lectura por bloques para limitar la cantidad de información cargada simultáneamente en memoria. Este enfoque permite trabajar con archivos de gran tamaño en un equipo local, aunque no proporciona las propiedades de escalabilidad horizontal de un motor distribuido.

En la implementación se utiliza un tamaño de bloque de 500.000 registros durante determinadas fases de lectura. Este valor constituye una decisión práctica del proyecto y no una recomendación general de *pandas*; su elección responde al equilibrio entre tiempo de procesamiento y consumo de memoria observado en el entorno utilizado.

Para el componente predictivo se ha utilizado *scikit-learn*. La biblioteca proporciona algoritmos supervisados, herramientas de evaluación y utilidades para construir flujos de aprendizaje automático. En concreto, se utiliza *RandomForestRegressor*, un estimador que

combina múltiples árboles de regresión y obtiene la predicción final mediante la agregación de sus resultados.

Power BI Desktop se utiliza como capa de visualización y análisis interactivo. Microsoft lo define como una aplicación de escritorio que permite conectarse a distintas fuentes, transformar información y crear informes visuales interactivos. También incorpora modelado de datos, relaciones, cálculos y diferentes tipos de visualización.

La selección de Power BI responde a que los resultados del pipeline se exportan en formatos compatibles y a que permite representar conjuntamente series temporales, mapas, tablas, indicadores y métricas predictivas. En este trabajo, Power BI no realiza el entrenamiento del modelo: actúa como capa final de exploración e interpretación.

Los archivos CSV se utilizan como formato de intercambio entre las distintas fases. Su ventaja en este contexto es la simplicidad, la facilidad de inspección y la compatibilidad con Python y Power BI. No obstante, esta elección también implica limitaciones frente a bases de datos o formatos columnares más eficientes para volúmenes mayores.

Aunque se estudió Apache Spark como alternativa de procesamiento distribuido y PostgreSQL como posible sistema de almacenamiento relacional, ninguno de los dos forma parte de la versión ejecutada. Por ello, deben presentarse como alternativas tecnológicas o líneas futuras y no como componentes de la arquitectura implementada.

Tecnología	Uso en el proyecto	Justificación
Python	Automatización del pipeline.	Integra procesamiento y modelado en un único entorno.
pandas	Lectura, limpieza, agregación y exportación.	Adecuado para datos tabulares y procesamiento local.
scikit-learn	Entrenamiento y evaluación.	Proporciona Random Forest y métricas de regresión.
Power BI	Informe interactivo.	Permite modelar y visualizar datos mediante gráficos, mapas y filtros.

CSV	Intercambio entre componentes.	Formato simple, trazable y compatible.
-----	--------------------------------	--

Tabla 2. Tecnologías seleccionadas para la implementación. Elaboración propia.

5.4.Fuentes de datos utilizadas

El caso práctico utiliza dos conjuntos de datos publicados por el Portal de Datos Abiertos del Ayuntamiento de Madrid.

La primera fuente es Tráfico. Histórico de datos del tráfico desde 2013, que contiene los datos históricos registrados en los puntos de medida y se actualiza mensualmente. El propio portal indica que cada mes se incorporan los datos correspondientes al mes anterior y proporciona documentación asociada sobre la estructura de los ficheros.

La segunda fuente es Tráfico. Ubicación de los puntos de medida del tráfico, relacionada directamente con el histórico. Este conjunto permite asociar los identificadores de los puntos de medida con información de localización y se publica en diferentes formatos, incluidos CSV y XLSX.

Fuentes	Formato	Periodo utilizado	Uso en el TFM
Histórico de datos del tráfico.	CSV mensual	2023 y 2024	Variables de tráfico y marca temporal.
Ubicación puntos de medida del tráfico.	Excel	Diciembre 2024	Información geográfica y descriptiva.

Tabla 3. Fuentes de datos utilizadas en el caso práctico. Elaboración propia.

Los archivos históricos utilizados incluyen los campos id, fecha, tipo_elem, intensidad, ocupacion, carga, vmed, error y periodo_integracion. La interpretación de estas variables debe realizarse conforme a la documentación oficial asociada al conjunto de datos y no únicamente a partir del nombre de la columna. El Ayuntamiento proporciona un documento específico de “contenido y estructura del fichero” dentro de la ficha del conjunto histórico.

- id: identificador del punto de medida.
- fecha: fecha y hora asociadas a la observación.
- tipo_elem: tipo de elemento de medida.
- intensidad: variable de intensidad de tráfico informada por el sistema. En la memoria final conviene reproducir la unidad exacta indicada en la documentación técnica oficial y mantenerla de forma consistente en todas las tablas y gráficos.
- ocupacion: indicador de ocupación asociado al punto de medida.
- carga: indicador relacionado con el grado de utilización de la vía.
- vmed: velocidad media asociada al registro.
- error: código o indicador que permite evaluar la validez del registro.
- periodo_integracion: periodo temporal utilizado para integrar la medición.

El conjunto geográfico incluye campos como id, tipo_elem, distrito, nombre, cod_cent, utm_x, utm_y, longitud y latitud. El campo id sirve como clave de enlace con los registros históricos. En sentido estricto, yo evitaría llamarlo “foreign key” en esta parte, porque los datos originales no forman una base de datos relacional con restricciones de clave foránea definidas. Es más preciso decir que se utiliza como campo común o clave de relación entre ambos conjuntos.

Las coordenadas longitud y latitud se utilizan posteriormente para construir las visualizaciones geográficas en Power BI, mientras que el distrito y el nombre permiten añadir contexto descriptivo a los puntos seleccionados.

5.5. Proceso de ingesta y caracterización de los datos

El proceso de ingesta se ha implementado mediante el script 01_ingesta_limpieza.py. Su función principal es localizar los ficheros mensuales de tráfico almacenados en el directorio de datos originales, leerlos de forma secuencial, seleccionar los puntos de medida incluidos en el caso de estudio y preparar los registros para las posteriores operaciones de limpieza y agregación.

La lectura de los ficheros históricos se realiza por bloques de 500.000 registros. Este mecanismo permite limitar la cantidad de información cargada simultáneamente en memoria y facilita el procesamiento de archivos de gran tamaño en un equipo local. En pandas, el parámetro chunksize permite precisamente leer un fichero de manera fragmentada en lugar

de cargarlo completamente en memoria, aunque este procedimiento sigue siendo un procesamiento local y no distribuido.

En total se han procesado 24 archivos mensuales, correspondientes a los doce meses de 2023 y los doce meses de 2024. La ejecución corregida del pipeline ha leído 308.458.911 registros originales y, tras aplicar el filtrado inicial sobre los puntos seleccionados y los registros válidos, ha conservado 3.339.435 observaciones para continuar con el proceso de transformación.

La validación se ha realizado sobre 50 puntos de medida. Esta selección permite mantener un volumen elevado de registros y realizar el procesamiento en el entorno local disponible. No obstante, los sensores fueron seleccionados conforme al criterio definido durante el desarrollo del pipeline y no mediante un muestreo estratificado por distrito, tipo de vía o características espaciales. Por tanto, no debe afirmarse que constituyen una muestra representativa de toda la red viaria de Madrid; su finalidad es proporcionar un conjunto de prueba suficientemente amplio para evaluar el funcionamiento de la solución.

Resumen cuantitativo del proceso de ingesta y limpieza:

- Ficheros mensuales procesados → 24
- Periodo → Enero de 2023 - diciembre de 2024
- Registros originales leídos → 308.458.911
- Registros válidos seleccionados → 3.339.435
- Puntos de medida → 50
- Observaciones horarias completas → 833.086
- Horas incompletas excluidas → 3.142

La diferencia entre los 3.339.435 registros seleccionados y las 833.086 observaciones horarias finales se debe principalmente a la agregación de mediciones con granularidad inferior a la hora y a la exclusión de las horas que no cumplen el criterio de completitud establecido.

5.6. Limpieza, transformación y agregación horaria

La fase de limpieza tiene como finalidad convertir los registros originales en un conjunto coherente y adecuado para el análisis. El proceso incluye la conversión del campo temporal,

la transformación de variables numéricas, la eliminación de registros sin identificador o sin fecha válida y el filtrado de observaciones señaladas como incorrectas por la fuente.

Posteriormente, los registros se agrupan por punto de medida y hora. Para considerar una hora como completa se exige disponer de las cuatro observaciones esperadas correspondientes a los minutos 00, 15, 30 y 45. Las horas que no cumplen este requisito se excluyen del dataset analítico. Con este procedimiento se han descartado 3.142 combinaciones sensor hora incompletas, obteniéndose finalmente 833.086 observaciones completas.

Esta condición de completitud es relevante porque evita calcular una intensidad horaria a partir de un número variable de mediciones. Además, permite que todas las observaciones incluidas en el dataset sigan un criterio homogéneo.

Una de las principales correcciones introducidas respecto a la versión anterior del proyecto afecta a la agregación de la intensidad. Inicialmente se sumaban las cuatro mediciones incluidas en una hora. Sin embargo, esta operación alteraba la escala de la variable, ya que las observaciones originales representan una medida de intensidad de tráfico y no cuatro conteos independientes que deban acumularse directamente. En la versión corregida se calcula la media de las cuatro observaciones de intensidad correspondientes a cada hora. En el archivo anterior todavía aparecía explícitamente la suma de intensidad.

Las demás variables operativas, como ocupación, carga y velocidad media, también se agregan mediante su media para cada combinación de sensor y hora. Asimismo, se conserva el número de registros utilizados, lo que permite comprobar que todas las observaciones finales proceden de cuatro mediciones válidas.

Problema o requisito	Tratamiento aplicado
Datos distribuidos en 24 ficheros	Lectura automatizada de todos los CSV
Volumen elevado	Lectura por bloques
Registros inválidos	Filtrado mediante el campo error
Fechas en formato no analítico	Conversión a fecha-hora

Variables numéricas como texto	Conversión explícita a formato numérico
Registros sin fecha o sensor	Eliminación
Intensidades negativas	Exclusión
Granularidad de 15 minutos	Agrupación por sensor y hora
Horas incompletas	Exclusión si no existen 4 observaciones
Variable intensidad	Media de las cuatro observaciones
Ausencia de geografía	Unión posterior con metadatos mediante id

Tabla 4. Principales controles y transformaciones. Elaboración propia.

5.7.Construcción del dataset analítico

El resultado del procesamiento es un dataset en el que cada fila representa una observación horaria completa para un punto de medida. La estructura incorpora información temporal, variables operativas y metadatos geográficos.

Las variables temporales permiten estudiar la evolución del tráfico según año, mes, día, hora y día de la semana. Las variables operativas describen las condiciones registradas en el sensor, mientras que las variables geográficas permiten localizar los puntos de medida y construir las visualizaciones espaciales.

Posteriormente, para el modelado predictivo se realiza una indexación horaria independiente para cada sensor. Este procedimiento genera una secuencia temporal completa y permite detectar huecos entre observaciones. Los intervalos sin datos permanecen como valores ausentes y no se rellenan artificialmente con observaciones de otros instantes.

Sobre esta secuencia se generan variables históricas específicas de cada sensor. Entre ellas se incluyen la intensidad de la hora anterior, la intensidad correspondiente a 24 horas antes y medias móviles calculadas únicamente a partir de observaciones previas. Esta forma de construcción evita que las variables históricas incorporen información perteneciente a la hora que se pretende estimar.

Grupo de variables	Variables incluidas
Identificación	id, nombre, distrito
Temporal	fecha_hora, año, mes, día, hora, día_semana, fin_semana
Tráfico	intensidad_horaria, ocupacion_media, carga_media, velocidad_media, registros_15min
Geografía	latitud, longitud
Históricas	Lag 1h, lag 24 h, medias móviles 3 h y 24 h

Tabla 5. Estructura del dataset analítico final. Elaboración propia.

La separación entre el dataset horario limpio y el conjunto utilizado por el modelo es importante. El primero contiene todas las observaciones completas disponibles, mientras que el segundo requiere además disponer de suficiente historia temporal para calcular los retardos y medias móviles. Por ello, no todas las observaciones horarias limpias son necesariamente elegibles para la predicción.

5.8. Modelado predictivo de intensidad del tráfico

El problema se formula como una tarea de regresión supervisada, ya que la variable objetivo es cuantitativa. Se pretende estimar la intensidad correspondiente a una hora determinada para un punto de medida utilizando exclusivamente información disponible con anterioridad.

Las variables explicativas incluyen características temporales, el identificador del sensor, retardos de intensidad, medias móviles e información operativa desplazada. Entre las variables históricas destacan `intensidad_lag_1h` e `intensidad_lag_24h`, además de medias móviles de tres y veinticuatro horas. Las variables de ocupación, carga y velocidad se utilizan con un retardo de una hora y no con sus valores de la propia hora objetivo.

Un modelo predictivo no debe utilizar variables que solo estarían disponibles después de producirse el instante que se pretende predecir. La documentación de scikit-learn advierte

que la fuga de datos puede producir estimaciones excesivamente optimistas del rendimiento cuando durante el entrenamiento se utiliza información no disponible en el escenario real.

El algoritmo seleccionado es RandomForestRegressor. El modelo se ha configurado con 100 árboles, una profundidad máxima de 14, un mínimo de cinco observaciones por hoja y una semilla fija para permitir la reproducibilidad de los resultados.

5.9.Estrategia de entrenamiento y evaluación

La evaluación se ha diseñado respetando el orden temporal de los datos. La documentación de scikit-learn señala que los problemas temporales requieren estrategias de partición que eviten entrenar con observaciones futuras y evaluar posteriormente sobre datos anteriores.

Se han utilizado tres periodos diferenciados:

- Entrenamiento inicial: enero–septiembre de 2023.
- Validación: octubre–diciembre de 2023.
- Prueba final: enero–diciembre de 2024.

Tras seleccionar y validar el modelo, se ha reentrenado utilizando todo el año 2023 y posteriormente se ha evaluado sobre las observaciones elegibles de 2024. El conjunto de prueba final contiene 371.128 observaciones.

Para interpretar correctamente el rendimiento se han utilizado dos modelos de referencia. El primero es un modelo de persistencia que utiliza la intensidad de la hora anterior (lag 1 h), mientras que el segundo utiliza la intensidad correspondiente a la misma hora del día anterior (lag 24 h). La comparación con ambos permite comprobar si el algoritmo proporciona una mejora respecto a reglas temporales sencillas.

Las métricas utilizadas son MAE, RMSE y R². El MAE expresa la magnitud media del error absoluto; el RMSE penaliza con mayor intensidad los errores elevados; y R² mide el grado en que las predicciones explican la variabilidad de la variable respecto a una referencia basada en su media. R² no debe interpretarse como un porcentaje de aciertos.

Modelo	MAE	RMSE	R ²
--------	-----	------	----------------

Baseline lag 24h	105,81	205,95	0,7685
Random Forest Regressor	52,48	89,29	0,9565

Tabla 6. Métricas de evaluación del modelo predictivo. Elaboración propia.

Los resultados muestran que Random Forest supera a ambos modelos de referencia en las tres métricas. El MAE se reduce aproximadamente un 40,5 % respecto a la persistencia de la hora anterior y alrededor de un 50,4 % respecto al baseline de 24 horas. El modelo también presenta un RMSE claramente inferior y un R^2 superior.

Estos resultados sustituyen a los valores anteriores de MAE 164,16, RMSE 311,83 y R^2 0,9666, ya que aquellos procedían de la versión del procesamiento que utilizaba una variable objetivo definida con una escala diferente. Los valores anteriores aparecen todavía en el documento que has enviado. Por tanto, no deben mezclarse ambas versiones ni compararse directamente como si utilizaran la misma variable objetivo.

En la fase de validación previa, el modelo obtuvo un MAE aproximado de 50,86, un RMSE de 92,56 y un R^2 de 0,9586, lo que muestra un comportamiento similar al observado posteriormente sobre el conjunto de prueba de 2024.

5.10. Importancia de las variables

La importancia calculada por Random Forest muestra que la mayor parte de la capacidad predictiva está asociada a la información histórica de intensidad.

Variable	Importancia aproximada
intensidad_lag_1h	86,09%
intensidad_lag_24h	8,65%
hora	1,75%
Media_movil_3h	1,11%
Resto de variables	<3% conjuntamente

Tabla 7. Variables más importantes según importancia. Elaboración propia.

La intensidad registrada una hora antes es claramente la variable más influyente. Esto indica una fuerte continuidad a corto plazo en la serie temporal. La intensidad de veinticuatro horas antes ocupa la segunda posición, lo que refleja la recurrencia de patrones diarios.

Debe tenerse en cuenta que estas importancias describen el comportamiento del modelo Random Forest y no establecen relaciones causales entre las variables y el tráfico. Asimismo, la variable correspondiente al número de registros de la hora tiene una importancia nula, ya que tras el filtro de calidad todas las observaciones incluidas contienen exactamente cuatro registros originales.

Las importancias antiguas donde `ocupacion_media` y `velocidad_media` aparecían entre las variables principales tampoco deben mantenerse, porque correspondían al modelo que utilizaba variables contemporáneas de la hora objetivo.

5.11. Análisis complementario de los errores

Además de las métricas globales, se ha realizado un análisis de la distribución de errores para comprobar si el rendimiento del modelo es homogéneo en el tiempo y entre sensores.

El error absoluto mediano es de aproximadamente 28,06 vehículos/hora, mientras que el percentil 90 se sitúa en 128,95 y el percentil 95 en 181,44. El error máximo alcanza valores considerablemente superiores, lo que evidencia la existencia de observaciones puntuales difíciles de predecir.

El error medio firmado, calculado como diferencia entre intensidad real y predicha, es de aproximadamente 0,07, por lo que no se observa un sesgo global importante hacia la sobreestimación o infraestimación.

El análisis mensual muestra que el MAE del modelo se mantiene por debajo del baseline de persistencia en todos los meses disponibles de 2024. No obstante, los meses de octubre y noviembre presentan menor cobertura de datos, por lo que sus errores más bajos deben interpretarse únicamente sobre las horas disponibles y con historia suficiente. No puede concluirse que el modelo funcione mejor durante los periodos en los que faltan observaciones.

También existen diferencias entre puntos de medida. El MAE mediano por sensor es de aproximadamente 48,38 vehículos/hora, aunque algunos puntos presentan errores notablemente superiores. Este análisis se incorpora al dashboard para identificar espacialmente los sensores cuya predicción resulta más difícil.

5.12. Diseño del informe interactivo en Power BI

La capa de visualización se ha implementado en Power BI a partir de los archivos generados por 03_export_powerbi.py. El script prepara tablas específicas para distintos tipos de análisis, incluyendo información temporal, geográfica, predictiva, métricas, importancia de variables, calidad y resúmenes generales.

Power BI permite utilizar filtros y segmentadores para modificar de forma interactiva los datos mostrados en los distintos objetos visuales. Microsoft indica que los segmentadores funcionan como controles de filtrado dentro de las páginas del informe y permiten explorar subconjuntos específicos de los datos.

La organización es la siguiente:

1. Portada. Presenta el título del proyecto y facilita la navegación.
2. Visión general. Resume los principales indicadores y la evolución global.
3. Calidad de datos. Muestra cobertura, registros procesados y problemas detectados.
4. Análisis temporal. Permite estudiar patrones horarios, semanales y mensuales.
5. Análisis geográfico. Representa los puntos de medida y permite comparar su comportamiento.
6. Modelo predictivo. Compara valores reales y predichos y presenta las métricas generales.
7. Análisis de errores. Analiza la magnitud y distribución de los errores por diferentes dimensiones.
8. Arquitectura y metodología. Resume visualmente las etapas del pipeline y las decisiones principales.

La incorporación de una página específica de calidad de datos y otra de análisis de errores responde a la necesidad de no limitar el informe a mostrar resultados finales. Estas páginas

permiten visualizar también la cobertura de la información y las limitaciones del modelo, proporcionando una interpretación más completa del funcionamiento de la arquitectura.

Es preferible denominar el resultado técnico “informe interactivo en Power BI” cuando quieras ser estricta con la terminología de Microsoft, ya que el archivo desarrollado contiene varias páginas. Puedes mantener “cuadro de mando” o “dashboard” como término general en el título o en la redacción, pero conviene explicar esta diferencia una vez.

5.13. Reutilización y posibilidades de ampliación

La arquitectura se ha diseñado separando las responsabilidades de adquisición, tratamiento, modelado y visualización. Esta organización facilita modificar un componente sin alterar necesariamente el resto de la solución.

Sin embargo, la reutilización en otros dominios no ha sido validada experimentalmente. Por tanto, no debe afirmarse que la arquitectura sea directamente reutilizable en energía, turismo o educación. Lo que sí puede defenderse es que su diseño modular ofrece una base susceptible de adaptación.

Para aplicar la solución a otro dominio sería necesario, como mínimo, modificar la adquisición de datos, redefinir las reglas de calidad, adaptar la variable objetivo y las características predictivas y modificar el modelo visual para representar indicadores específicos del nuevo contexto.

De igual forma, una ampliación hacia mayores volúmenes o procesamiento continuo podría requerir sustituir algunos componentes locales por tecnologías distribuidas, bases de datos u otros formatos de almacenamiento. Apache Spark, PostgreSQL, almacenamiento cloud y procesamiento en streaming se consideran posibles alternativas futuras, pero no forman parte de la solución validada en este TFM.

En consecuencia, la generalización de la arquitectura debe formularse como una posibilidad de diseño y una línea de trabajo futura, mientras que las conclusiones del presente proyecto deben limitarse al escenario de tráfico de Madrid y al entorno local realmente evaluado.

6. Código fuente y datos analizados

Este capítulo documenta la organización del código fuente, los archivos de configuración y los datos utilizados durante la implementación. Su finalidad es complementar el desarrollo técnico descrito en el capítulo anterior y proporcionar la información necesaria para comprender la estructura del proyecto y reproducir sus principales etapas.

La organización del proyecto separa el código, los datos originales, los datos procesados y los elementos destinados a la visualización. Esta separación permite mantener la trazabilidad entre los datos de entrada y los resultados generados y evita modificar directamente los archivos originales durante las sucesivas transformaciones.

Además de los scripts de procesamiento y modelado, la versión final incorpora documentación del entorno, dependencias y pruebas. Estos elementos resultan especialmente relevantes en un trabajo de desarrollo software, ya que la guía de UNIR establece que la contribución debe materializarse en una herramienta funcional susceptible de demostración práctica.

6.1. Organización del repositorio

El proyecto se organiza bajo una carpeta principal que contiene de manera diferenciada el código fuente, los datos, los resultados destinados a Power BI y la documentación auxiliar.

Una estructura simplificada del proyecto es la siguiente:

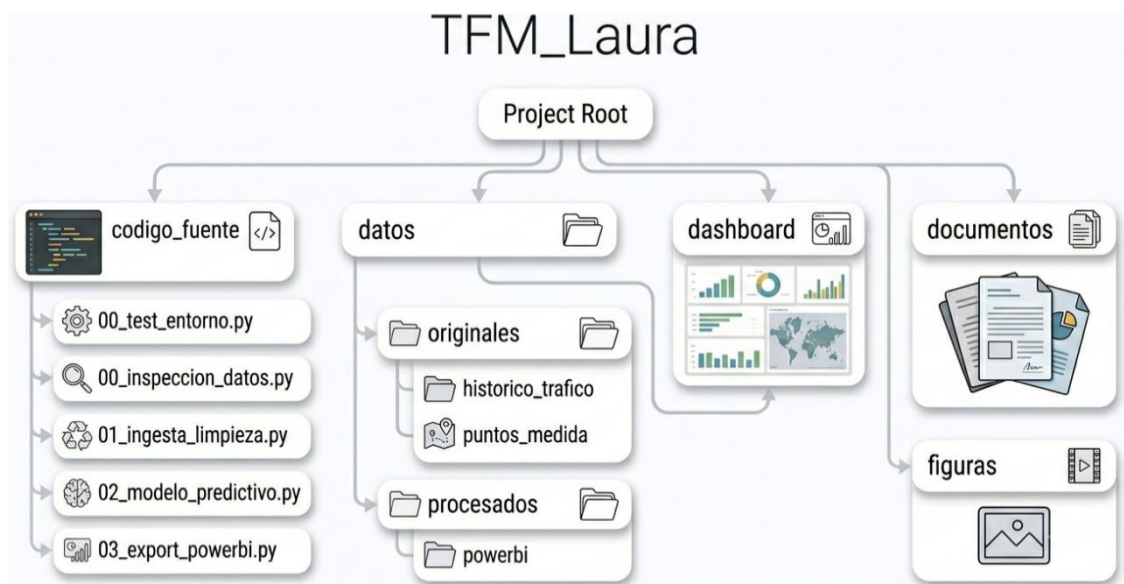


Figura 3. Organización del repositorio donde se encuentra el proyecto. Elaboración IA

La carpeta de datos originales se mantiene separada de la carpeta de resultados procesados. De esta forma, los archivos descargados de las fuentes oficiales permanecen sin modificar, mientras que las transformaciones sucesivas se almacenan en directorios específicos.

La utilización de una estructura de directorios estable también facilita que los scripts puedan localizar automáticamente sus entradas y escribir sus salidas en rutas predefinidas, reduciendo la necesidad de modificar manualmente las rutas durante cada ejecución.

6.2. Scripts desarrollados

El código se ha dividido en diferentes scripts con responsabilidades específicas. Esta organización permite ejecutar las fases por separado y facilita la localización de errores y la repetición de procesos concretos.

Script	Función
00_test_entorno.py	Comprueba que el entorno virtual y las bibliotecas principales están correctamente instaladas.
00_inspeccion_datos.py	Inspecciona las columnas y primeras filas de los ficheros originales para adaptar el pipeline a la estructura real de los datos.
01_ingesta_limpieza.py	Lee los ficheros mensuales por bloques, filtra registros erróneos, agrega los datos a nivel horario y une la información geográfica.
02_modelo_predictivo.py	Genera variables históricas, divide el dataset en entrenamiento y prueba, entrena el modelo Random Forest y exporta métricas y predicciones.
03_export_powerbi.py	Prepara tablas específicas para Power BI, incluyendo datos temporales, puntos, predicciones, métricas, calidad de datos e importancia de variables.

Tabla 8. Scripts desarrollados durante el proyecto. Elaboración propia

El script `01_ingesta_limpieza.py` realiza el procesamiento principal de los datos originales. Entre otras operaciones, comprueba duplicados, transforma las variables, construye observaciones horarias y excluye las horas que no contienen las cuatro mediciones previstas.

El script `02_modelo_predictivo.py` añade las variables necesarias para el análisis predictivo. En la versión corregida utiliza retardos temporales calculados por sensor y separa cronológicamente los datos en entrenamiento, validación y prueba. También calcula dos modelos de referencia y el Random Forest utilizado como modelo principal.

El script `03_export_powerbi.py` genera tablas separadas para distintos objetivos visuales. Esta decisión evita depender de una única tabla de gran tamaño para todas las páginas del informe y permite disponer de archivos específicos para métricas, predicciones, calidad, información temporal y datos geográficos.

6.3. Documentación y dependencias

Además del código fuente, el repositorio contiene un archivo `requirements.txt` en el que se especifican las dependencias necesarias para ejecutar los scripts. Este mecanismo permite documentar las bibliotecas utilizadas y facilita la recreación del entorno de ejecución.

Python permite aislar las dependencias de un proyecto mediante entornos virtuales, evitando interferencias entre las bibliotecas requeridas por diferentes aplicaciones.

El archivo `README.md` complementa esta información mediante una descripción de la estructura del proyecto y del orden recomendado de ejecución de los scripts. Para garantizar la reproducibilidad, la documentación final deberá indicar como mínimo los requisitos del entorno, las rutas esperadas, los archivos originales necesarios y la secuencia de ejecución.

6.4. Datos originales utilizados

Los datos originales se almacenan bajo `datos/originales` y se mantienen sin modificaciones para conservar una referencia directa a las fuentes descargadas.

La subcarpeta `historico_trafico` contiene los 24 archivos mensuales correspondientes a enero de 2023–diciembre de 2024. Estos archivos constituyen la entrada principal del proceso de ingesta.

La subcarpeta `puntos_medida` contiene el fichero geográfico utilizado para relacionar los identificadores de los sensores con información descriptiva y espacial, como el distrito, el nombre del punto, la longitud y la latitud.

Los datos proceden del Portal de Datos Abiertos del Ayuntamiento de Madrid. La procedencia, estructura de las variables y condiciones de reutilización se han descrito previamente en los capítulos 4 y 5, por lo que no resulta necesario reproducir aquí nuevamente toda la descripción de las fuentes.

6.5. Datos procesados generados

La ejecución de la versión corregida del pipeline genera diferentes archivos para separar los resultados de limpieza, modelado y visualización. Todos ellos se almacenan bajo la versión v2, evitando sobrescribir los resultados correspondientes a iteraciones anteriores.

La principal salida del procesamiento es el conjunto horario limpio, que contiene 833.086 observaciones completas correspondientes a 50 puntos de medida durante 2023 y 2024. Esta cifra sustituye a los 836.228 registros indicados en la versión anterior.

Archivo generado	Descripción
<code>trafico_limpio.csv</code>	Dataset horario final con 833.086 registros.
<code>resumen_limpieza.csv</code>	Resumen por fichero mensual con registros leídos, válidos y horarios generados.
<code>puntos_medida_limpios.csv</code>	Tabla depurada de puntos de medida.
<code>puntos_seleccionados_modelo.csv</code>	Lista de los 50 puntos seleccionados para el prototipo.
<code>trafico_modelo.csv</code>	Dataset con variables adicionales para modelado predictivo.

predicciones.csv	Valores reales, predicciones del baseline, predicciones del Random Forest y errores.
metricas_modelo.csv	Métricas MAE, RMSE y R^2 de los modelos evaluados.
importancia_variables.csv	Importancia de las variables según el modelo Random Forest.
resumen_modelo.csv	Información resumida sobre entrenamiento, validación y prueba.

Tabla 9. Archivos procesados generados por la arquitectura. Elaboración propia.

El dataset limpio no coincide exactamente con el utilizado por el modelo. Para generar determinadas variables históricas es necesario disponer de observaciones anteriores, por lo que las filas sin suficiente historia temporal quedan excluidas de la muestra predictiva. Esta diferencia explica que el conjunto de prueba de 2024 contenga 371.128 observaciones elegibles y no la totalidad de las observaciones horarias disponibles.

6.6. Archivos destinados a Power BI

El script 03_export_powerbi.py genera un conjunto independiente de archivos preparados específicamente para la visualización. En la versión final se utilizan los siguientes:

- trafico_powerbi.csv
- trafico_temporal_powerbi.csv
- trafico_puntos_powerbi.csv
- predicciones_powerbi.csv
- metricas_powerbi.csv
- importancia_variables_powerbi.csv
- resumen_limpieza_powerbi.csv
- resumen_general_powerbi.csv
- calidad_datos_powerbi.csv
- puntos_medida_powerbi.csv

Estos archivos se exportan con una configuración homogénea de separador, codificación y formato decimal para facilitar su importación en Power BI.

La separación en varias tablas responde a necesidades analíticas diferentes. Por ejemplo, las predicciones y métricas se utilizan en las páginas relacionadas con el modelo, mientras que la información de cobertura y limpieza alimenta las visualizaciones de calidad de datos.

Power BI permite importar datos desde archivos de texto y CSV y utilizarlos como origen para modelos e informes interactivos.

6.7. Disponibilidad del código y los datos

El código fuente utilizado para el desarrollo del proyecto, junto con la documentación de ejecución y las dependencias, se encuentra disponible en el repositorio indicado en https://github.com/lauracm2002/TFM_Laura. Los datos originales no se redistribuyen necesariamente dentro del repositorio, sino que se proporciona la referencia a las fuentes oficiales y las instrucciones necesarias para obtenerlos.

.

7. Conclusiones

7.1. Conclusiones del proyecto

El desarrollo realizado ha permitido diseñar, implementar y evaluar una arquitectura modular orientada al procesamiento de datos históricos, el análisis predictivo y la visualización interactiva. La solución se ha aplicado a un caso práctico de movilidad urbana basado en datos abiertos de tráfico de Madrid, integrando en un mismo flujo las etapas de adquisición, limpieza, construcción del dataset analítico, modelado y explotación visual.

Una de las principales aportaciones del proyecto es la construcción de un pipeline reproducible para la ingesta y preparación de datos. A partir de 24 ficheros mensuales correspondientes a 2023 y 2024 y un volumen original de 308.458.911 registros, el proceso de limpieza y agregación ha generado un dataset final de 833.086 observaciones horarias completas correspondientes a 50 puntos de medida. Esta transformación ha permitido pasar de registros con granularidad inferior a la hora y distribuidos en múltiples archivos a una estructura homogénea y adecuada para el análisis.

La revisión metodológica realizada durante el desarrollo ha sido especialmente relevante. La versión inicial agregaba la intensidad mediante suma y utilizaba variables contemporáneas de la hora objetivo, lo que podía alterar la escala de la variable y producir fuga de información. En la versión corregida, la intensidad horaria se calcula mediante la media de las cuatro observaciones válidas de cada hora, y las variables predictivas se construyen únicamente con información disponible previamente. Este ajuste ha permitido formular un escenario predictivo más coherente con una utilización realista del modelo.

Desde el punto de vista predictivo, el modelo Random Forest Regressor ha obtenido resultados superiores a los dos modelos de referencia utilizados. Sobre el conjunto de prueba de 2024, el modelo ha alcanzado un MAE de 52,48 vehículos/hora, un RMSE de 89,29 vehículos/hora y un R^2 de 0,9565. Frente al baseline de persistencia de una hora, cuyo MAE es de 88,14 vehículos/hora, el Random Forest reduce el error absoluto medio en aproximadamente un 40,5 %. Frente al baseline de 24 horas, con un MAE de 105,81 vehículos/hora, la reducción es de aproximadamente un 50,4 %.

La importancia de variables muestra que la intensidad de la hora anterior concentra la mayor parte de la capacidad predictiva del modelo, con aproximadamente un 86,09 % de importancia, seguida de la intensidad de 24 horas antes, con alrededor de un 8,65 %. Estos resultados indican una fuerte dependencia temporal a corto plazo y una recurrencia diaria del comportamiento del tráfico. No obstante, estas importancias deben interpretarse como una medida interna del modelo y no como relaciones causales.

La evaluación también ha mostrado que el comportamiento del modelo no es completamente homogéneo entre periodos y sensores. El error absoluto mediano es de aproximadamente 28,06 vehículos/hora, mientras que determinados puntos de medida presentan errores considerablemente mayores. Asimismo, los meses con menor cobertura de datos deben interpretarse con cautela, ya que las métricas solo representan las horas para las que existe información suficiente y no permiten inferir el rendimiento durante los periodos ausentes.

La capa de visualización completa la arquitectura mediante un informe interactivo en Power BI compuesto por ocho páginas: Portada, Visión general, Calidad de datos, Análisis temporal, Análisis geográfico, Modelo predictivo, Análisis de errores y Arquitectura y metodología. Esta estructura permite explorar no solo el comportamiento del tráfico y las predicciones, sino también la calidad de los datos y las limitaciones del modelo.

En conjunto, el TFM demuestra que una arquitectura modular basada en herramientas accesibles puede proporcionar una solución funcional para el análisis de datos temporales y georreferenciados en un entorno local. Sin embargo, los resultados obtenidos deben interpretarse dentro del alcance realmente evaluado: una selección de 50 puntos de medida, procesamiento por lotes y ejecución local. La escalabilidad distribuida, el procesamiento continuo y la aplicación efectiva a otros dominios no se han validado experimentalmente y se consideran líneas de ampliación futura.

7.2.Relación entre objetivos y resultados obtenidos

Descripción	Resultado obtenido
-------------	--------------------

Analizar requisitos técnicos de una arquitectura de datos masivos.	Se revisaron las necesidades de ingesta, procesamiento, modelado y visualización.
Seleccionar e integrar fuentes abiertas de tráfico urbano.	Se utilizaron histórico de tráfico y puntos de medida del Ayuntamiento de Madrid.
Diseñar una arquitectura modular.	Se definieron capas de adquisición, ingesta, procesamiento, modelado y visualización.
Implementar limpieza y transformación reproducible.	Se desarrolló un pipeline Python que generó 833.086 registros horarios.
Desarrollar y evaluar modelos predictivos.	Se comparó baseline lag 24h con Random Forest, obteniendo $R^2=0,9565$.
Diseñar un dashboard interactivo.	Se creó un informe Power BI con cuatro páginas analíticas.
Valorar utilidad, limitaciones y reutilización.	Se documentaron limitaciones, trabajo futuro y posibles dominios de reutilización.

Tabla 11. Relación entre los objetivos específicos y los resultados obtenidos. Elaboración propia.

7.3. Conclusiones personales y aprendizaje adquirido

Desde un punto de vista académico y técnico, el desarrollo de este TFM me ha permitido consolidar conocimientos relacionados con el ciclo completo de un proyecto de datos, desde la adquisición y organización de fuentes abiertas hasta la construcción de un modelo predictivo y su explotación mediante visualización interactiva.

Uno de los principales aprendizajes ha sido comprender que la calidad del modelo depende en gran medida de las decisiones tomadas durante la preparación de los datos. La revisión del proceso de agregación, la detección de horas incompletas y la corrección de la fuga de información mostraron que una buena métrica no es suficiente si la definición de la variable objetivo o el protocolo de evaluación no son metodológicamente correctos.

También he profundizado en el uso de Python y pandas para automatizar tareas de ingesta, limpieza y transformación, así como en scikit-learn para la construcción y evaluación de modelos supervisados. El proyecto me ha permitido comprobar la importancia de respetar el orden temporal de los datos, utilizar modelos de referencia y analizar los errores de forma complementaria a las métricas globales.

En cuanto a la visualización, el desarrollo del informe en Power BI me ha permitido comprender la relevancia de estructurar la información de manera que los resultados técnicos puedan ser interpretados desde distintas perspectivas. La inclusión de páginas específicas de calidad de datos y análisis de errores ha reforzado la idea de que una visualización útil no debe mostrar únicamente resultados favorables, sino también las limitaciones y condiciones bajo las que esos resultados han sido obtenidos.

Otro aprendizaje relevante ha sido la necesidad de adaptar las decisiones tecnológicas al alcance real del proyecto. El procesamiento por bloques mediante pandas ha permitido trabajar con un volumen elevado de información en un entorno local, pero también ha puesto de manifiesto las diferencias entre una solución local y una arquitectura distribuida. Esta distinción ha sido importante para evitar atribuir al sistema capacidades de escalabilidad que no han sido demostradas experimentalmente.

Finalmente, el proyecto me ha permitido integrar programación, análisis de datos, aprendizaje automático y visualización dentro de una única solución. Más allá de las herramientas concretas utilizadas, el principal aprendizaje ha sido la importancia de mantener la trazabilidad entre los datos de entrada, las transformaciones realizadas, las predicciones generadas y los resultados presentados. Esta visión integral del proceso constituye una de las principales aportaciones formativas del trabajo.

8. Limitaciones y prospectiva

8.1. Limitaciones

A pesar de los resultados obtenidos, la solución presenta una serie de limitaciones que deben tenerse en cuenta para interpretar correctamente el alcance del trabajo. Estas restricciones afectan tanto a la infraestructura utilizada como a los datos, el modelo predictivo y la capa de visualización.

La primera limitación está relacionada con el entorno de ejecución. La arquitectura se ha desarrollado y evaluado de forma local utilizando Python, pandas, archivos CSV, scikit-learn y Power BI Desktop. Aunque la lectura por bloques permite controlar el consumo de memoria al procesar ficheros de gran tamaño, este mecanismo continúa siendo un procesamiento local y no proporciona las características de una infraestructura distribuida. Por tanto, los resultados obtenidos no permiten afirmar que la solución sea escalable horizontalmente ni equivalente a una plataforma Big Data basada en clústeres.

Las plataformas distribuidas como Azure Databricks se apoyan en Apache Spark y están diseñadas para ejecutar cargas de trabajo de ingeniería de datos, análisis y aprendizaje automático a escala. Su utilización implica, no obstante, decisiones adicionales relacionadas con infraestructura, costes, seguridad y gobierno, por lo que su idoneidad debe evaluarse según las necesidades del proyecto.

La segunda limitación corresponde a la selección de los puntos de medida. Para construir el caso práctico se han utilizado 50 sensores, seleccionados atendiendo a criterios de disponibilidad de datos definidos durante el desarrollo. Esta selección no corresponde a un muestreo espacial estratificado por distrito, tipo de vía, intensidad o localización geográfica. En consecuencia, los resultados obtenidos describen los puntos incluidos en el análisis, pero no deben interpretarse como una representación estadísticamente garantizada de toda la red de tráfico de Madrid.

Una tercera limitación procede de la propia cobertura de los datos. Aunque se han utilizado registros correspondientes a 2023 y 2024, existen periodos con menor disponibilidad de observaciones, especialmente en octubre y noviembre de 2024. El proceso de limpieza evita

rellenar artificialmente estos huecos y excluye las observaciones que no cumplen los requisitos definidos, pero esta decisión reduce la cobertura disponible para el análisis.

Esta circunstancia también afecta a la evaluación predictiva. Las métricas del modelo se calculan únicamente sobre las horas para las que existe información suficiente para construir las variables históricas necesarias. Por ello, un buen resultado sobre las observaciones evaluables no implica que el modelo pueda generar predicciones fiables durante los periodos en los que faltan datos anteriores.

Otra limitación se encuentra en las variables utilizadas para el modelado. La versión final incorpora información temporal, retardos de intensidad, medias móviles y variables operativas anteriores, pero no utiliza factores externos como meteorología, festivos, accidentes, obras, eventos o incidencias. Estos elementos pueden estar relacionados con cambios puntuales en el comportamiento del tráfico, por lo que sería necesario incorporarlos y evaluarlos experimentalmente antes de determinar si aportan una mejora.

El modelo predictivo también presenta una limitación de alcance metodológico. Random Forest Regressor constituye el algoritmo principal evaluado frente a dos estrategias de referencia temporales, pero no se ha realizado una comparación exhaustiva con una amplia variedad de modelos. Por tanto, los resultados permiten concluir que Random Forest supera a los baselines utilizados, pero no que sea el mejor algoritmo posible para este conjunto de datos.

Asimismo, el planteamiento predictivo supone que, en el instante de generar una predicción, está disponible la información correspondiente a horas anteriores, incluidas las variables operativas retardadas. Esta condición se cumple al evaluar sobre el histórico, pero no se ha desplegado una infraestructura que compruebe su disponibilidad efectiva en tiempo real. En consecuencia, el proyecto demuestra un escenario predictivo histórico de una hora de anticipación, pero no un servicio operativo de predicción en producción.

Otra limitación afecta a la validación del software. Las pruebas automatizadas realizadas permiten verificar determinadas reglas de transformación sobre casos controlados, pero no equivalen a una evaluación exhaustiva de integración, rendimiento, estrés o escalabilidad. Del mismo modo, si la versión final no incorpora una prueba formal de usabilidad o aceptación

con usuarios, esta ausencia debe señalarse expresamente, ya que el hecho de que el informe sea funcional no demuestra por sí mismo su facilidad de uso.

Finalmente, la capa de visualización se ha desarrollado en Power BI Desktop como un informe interactivo. No se ha desplegado el sistema en Power BI Service ni se ha configurado una actualización automática conectada a nuevas fuentes. Por tanto, el informe representa una capa de exploración del histórico y de los resultados previamente generados, no un sistema operativo continuamente actualizado.

8.2.Trabajo futuro

Una primera línea de ampliación consistiría en aumentar la cobertura temporal y espacial del análisis. El Portal de Datos Abiertos del Ayuntamiento de Madrid dispone de históricos anteriores a 2023, por lo que podría ampliarse el periodo analizado e incluir un número mayor de puntos de medida. En este caso sería recomendable utilizar un procedimiento de selección que garantice una cobertura espacial más equilibrada o, si los recursos lo permiten, procesar la totalidad de los sensores disponibles.

Una segunda línea de trabajo consiste en enriquecer el dataset con variables externas. Podrían incorporarse datos meteorológicos, calendarios de días festivos, información sobre obras, eventos o incidencias de tráfico. La hipótesis sería que estos factores ayudan a explicar comportamientos que no pueden capturarse únicamente mediante patrones históricos, pero su utilidad debería comprobarse mediante una nueva evaluación y no asumirse de antemano.

También sería posible ampliar la comparación de algoritmos. Entre las alternativas pueden analizarse métodos de Gradient Boosting, XGBoost, modelos estadísticos de series temporales o arquitecturas de aprendizaje profundo. La comparación debería realizarse utilizando el mismo horizonte predictivo, las mismas observaciones y las mismas particiones temporales para mantener la comparabilidad de las métricas.

Desde el punto de vista de la infraestructura, una evolución relevante sería evaluar tecnologías de procesamiento distribuido. Azure Databricks, por ejemplo, se basa en Apache Spark y permite ejecutar cargas de trabajo de procesamiento por lotes y streaming a escala.

Una posible arquitectura futura podría separar el almacenamiento en distintas capas —datos originales, datos validados y datos preparados para consumo— e incorporar un motor distribuido para las transformaciones. Microsoft describe este tipo de organización mediante capas de datos sin procesar, limpiados y preparados para el consumo analítico.

Otra evolución consistiría en automatizar la adquisición de nuevos datos y aproximar la arquitectura a un funcionamiento continuo. Una infraestructura de streaming permitiría ingerir eventos a medida que se generan, procesarlos y almacenarlos para análisis posterior. Las arquitecturas de referencia de Microsoft para Databricks muestran flujos que separan ingesta, procesamiento, almacenamiento y análisis sobre datos recibidos de forma continua.

No obstante, la migración hacia una infraestructura distribuida o cloud debería evaluarse también en términos de coste, complejidad y necesidades reales. El hecho de que una tecnología admita procesamiento distribuido no implica automáticamente que resulte más adecuada para el volumen y frecuencia de actualización de cualquier proyecto.

En la capa de visualización, una evolución natural sería publicar el informe mediante Power BI Service. Este entorno permite organizar informes y modelos semánticos en áreas de trabajo, asignar diferentes roles de acceso y facilitar la colaboración entre usuarios.

También podría configurarse una actualización programada de los datos. Power BI Service permite definir programaciones de actualización cuando se dispone de conectividad compatible con los orígenes utilizados, además de notificar errores producidos durante este proceso.

Otra posible ampliación sería incorporar alertas sobre determinados indicadores. Power BI Service permite establecer alertas cuando el valor de determinados elementos, como tarjetas o KPI, supera o desciende por debajo de umbrales definidos, siempre que el modelo de datos se actualice.

Estas mejoras permitirían evolucionar el prototipo hacia una solución con un mayor grado de automatización y disponibilidad, aunque requerirían una nueva fase de diseño, implementación y evaluación antes de poder considerarse capacidades demostradas.

8.3. Prospectiva de la arquitectura

A medio plazo, la arquitectura desarrollada puede considerarse una base sobre la que experimentar con nuevas fuentes de datos, tecnologías de procesamiento y estrategias predictivas. Su separación en componentes facilita identificar qué módulos deberían sustituirse o ampliarse según las necesidades del escenario.

Una posible evolución sería conservar la lógica general del pipeline —adquisición, preparación, modelado, evaluación y visualización— y reemplazar progresivamente determinados componentes locales por servicios distribuidos o administrados. Esta transición permitiría analizar de manera experimental hasta qué punto las decisiones tomadas para el prototipo continúan siendo válidas al aumentar el volumen, la frecuencia de actualización o el número de usuarios.

La posible aplicación a otros dominios también forma parte de esta prospectiva. La organización modular podría adaptarse a conjuntos de datos temporales relacionados con energía, transporte público, ocupación o demanda de servicios. No obstante, cada nuevo dominio requeriría redefinir sus fuentes, reglas de calidad, variables objetivo, criterios de evaluación y visualizaciones. Por ello, la generalización debe considerarse una hipótesis de diseño y no una capacidad validada por los resultados actuales.

En conjunto, las limitaciones identificadas no invalidan los resultados obtenidos, sino que delimitan el contexto en el que deben interpretarse. El trabajo demuestra el funcionamiento de una arquitectura modular aplicada a históricos de tráfico de Madrid en un entorno local; su evolución hacia una solución distribuida, automatizada, continuamente actualizada y validada con usuarios constituye el principal ámbito de desarrollo futuro.

Referencias bibliográficas

Kitchin, R. (2014). Big Data, new epistemologies and paradigm shifts. Big Data & Society, 1(1). <https://doi.org/10.1177/2053951714528481>

Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5-32. <https://doi.org/10.1023/a:1010933404324>

Tráfico. Histórico de datos del tráfico desde 2013 - Portal de datos abiertos del Ayuntamiento de Madrid. (s. f.). <https://datos.madrid.es/dataset/208627-0-transporte-ptomedida-historico>

Tráfico. Ubicación de los puntos de medida del tráfico - Portal de datos abiertos del Ayuntamiento de Madrid. (s. f.-b). <https://datos.madrid.es/dataset/202468-0-intensidad-trafico>

Jonker, T. K. A. (2026, 28 mayo). ¿Qué es la arquitectura de datos? IBM. <https://www.ibm.com/es-es/think/topics/data-architecture>

Kosinski, A. B. M. (2026, 27 mayo). ¿Qué es el big data? IBM. <https://www.ibm.com/es-es/think/topics/big-data>

Vibhareddyv. (s. f.). Arquitecturas de macrodatos - Azure Architecture Center. Microsoft Learn. <https://learn.microsoft.com/es-es/azure/architecture/databases/guide/big-data-architectures>

Kosinski, A. J. J. H. M. (2026, 27 mayo). Qué es un almacén de datos. <https://www.ibm.com/es-es/think/topics/data-warehouse>

Kosinski, M. (2025, 28 noviembre). Almacenes de datos vs. data lakes vs. lakehouses de datos. <https://www.ibm.com/es-es/think/topics/data-warehouse-vs-data-lake-vs-data-lakehous>

Breiman, L. (2001b). Random forests. Machine Learning, 45(1), 5-32. <https://doi.org/10.1023/a:1010933404324>

Common pitfalls and recommended practices. (s. f.). Scikit-learn. https://scikit-learn.org/stable/common_pitfalls.html

TimeSeriesSplit. (s. f.). Scikit-learn. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.TimeSeriesSplit.html

JulCsc. (s. f.). Informes en el servicio Power BI - Power BI. Microsoft Learn. <https://learn.microsoft.com/es-es/power-bi/explore-reports/end-user-reports>

JulCsc. (s. f.-b). Personalización del panel Visualizaciones en Power BI Desktop y el servicio Power BI - Power BI. Microsoft Learn. <https://learn.microsoft.com/es-es/power-bi/visuals/power-bi-report-visualizations>

JulCsc. (s. f.-b). Introducción a Power BI Desktop - Power BI. Microsoft Learn. <https://learn.microsoft.com/es-es/power-bi/fundamentals/desktop-getting-started>

JulCsc. (s. f.-b). Introducción a los paneles para los diseñadores de Power BI - Power BI. Microsoft Learn. <https://learn.microsoft.com/es-es/power-bi/create-reports/service-dashboards>

pandas documentation — pandas 3.0.5 documentation. (s. f.). <https://pandas.pydata.org/docs/>

Haya, P. (2022, 5 enero). La metodología CRISP-DM en ciencia de datos. Instituto de Ingeniería del Conocimiento. <https://www.iic.uam.es/innovacion/metodologia-crisp-dm-ciencia-de-datos/>

BOE-A-2007-19814 Ley 37/2007, de 16 de noviembre, sobre reutilización de la información del sector público. (s. f.-b). <https://www.boe.es/buscar/act.php?id=BOE-A-2007-19814>

Condiciones de uso - Portal de datos abiertos del Ayuntamiento de Madrid. (s. f.). <https://datos.madrid.es/pages/condiciones-de-uso>

BOE-A-2025-24545 Ley 9/2025, de 3 de diciembre, de Movilidad Sostenible. (s. f.). <https://www.boe.es/buscar/act.php?id=BOE-A-2025-24545>

David, J. (2025, 14 julio). Top 10 clean code practices to reduce technical debt. Sparity. <https://www.sparity.com/blogs/the-importance-of-business-intelligence/>

Amazon Web Services. (s. f.). ¿Qué es extracción, transformación y carga (ETL)? Amazon Web Services, Inc. <https://aws.amazon.com/es/what-is/etl/>

Ibm. (2026, 2 junio). Calidad de los datos. IBM. <https://www.ibm.com/es-es/think/topics/data-quality>

Capdevila, F. (2026, 22 abril). ► ¿Qué es Power BI? Guía completa: para qué sirve, cómo funciona y precios (2026). Hako IT. <https://www.hakoit.com/que-es-power-bi/>

Cerveró, E. (2026, 21 enero). ¿Qué es Power BI, para qué sirve y cómo funciona? Aglaia. <https://aglaia.es/blog/power-bi/que-es-power-bi-para-que-sirve-y-como-funciona/>

Estudio de la movilidad con Big Data | Ministerio de Transportes y Movilidad Sostenible. (s. f.). <https://www.transportes.gob.es/ministerio/proyectos-singulares/estudio-de-movilidad-con-big-data>

LA REVOLUCIÓN DEL BIG DATA EN EL TRANSPORTE: TRANSFORMANDO LA MOVILIDAD EN LA ERA DIGITAL. (s. f.). Master Big Data y Data Science UNED. <https://www.masterbigdataonline.com/blog/20250109-big-data-sector-transportes.html>

Martínez, S. (2026, 30 enero). Ciclo de vida de los datos: qué es, etapas y cómo gestionarlo correctamente. Inesdi. <https://www.inesdi.com/blog/ciclo-vida-datos/>

JulCsc. (s. f.). Introducción a Power BI Desktop - Power BI. Microsoft Learn. <https://learn.microsoft.com/es-es/power-bi/fundamentals/desktop-getting-started>

Ciclo de vida de los datos - AWS Orientación prescriptiva. (s. f.). https://docs.aws.amazon.com/es_es/prescriptive-guidance/latest/modern-data-centric-use-cases/data-lifecycle.html

Hai, R., Koutras, C., Quix, C., & Jarke, M. (2023). Data Lakes: A Survey of Functions and Systems. IEEE Transactions On Knowledge And Data Engineering, 35(12), 12571-12590. <https://doi.org/10.1109/tkde.2023.3270101>

Bienvenido - EMT data. (s. f.). <https://datos.emtmadrid.es/>

Levin, W. L. C. D. B. O. (2020, 7 enero). NIST Big Data Interoperability Framework: Volume 6, Reference Architecture. NIST. <https://www.nist.gov/publications/nist-big-data-interoperability-framework-volume-6-reference-architecture>

JulCsc. (s. f.-b). Power BI documentation - Power BI. Microsoft Learn. <https://learn.microsoft.com/en-us/power-bi/>

Anexo A. Privacidad y protección de datos

Este anexo tiene como finalidad analizar si los datos utilizados en el Trabajo Fin de Máster implican tratamiento de datos personales y, en consecuencia, si resulta aplicable la normativa de protección de datos personales.

El TFM propone el diseño e implementación de una arquitectura general de datos masivos para el análisis predictivo y la visualización interactiva, validada mediante un caso práctico de movilidad urbana. Para ello, se prevé utilizar datos abiertos procedentes de fuentes institucionales, preferentemente en formato agregado, anonimizado o sin identificadores personales.

A.1. Tipo de datos tratados

Los datos previstos para la validación de la arquitectura estarán relacionados con movilidad urbana, como registros agregados de tráfico, puntos de medición, intensidad de circulación, franjas temporales, localización de sensores o información de transporte publicada en portales de datos abiertos.

No se prevé tratar datos personales identificados o identificables, como nombres, DNI, matrículas, identificadores de dispositivos móviles, trayectorias individuales o datos que permitan reconstruir el comportamiento de una persona concreta.

La Agencia Española de Protección de Datos indica que el RGPD no afecta al tratamiento de información anónima cuando esta no guarda relación con una persona física identificada o identificable, o cuando los datos se han anonimizado de forma que la persona no pueda ser identificada. También señala que debe valorarse si existe una probabilidad razonable de identificación.

A.2. Aplicación o exclusión de la normativa de protección de datos

En función del tipo de datos previstos, este TFM se acoge al procedimiento de exclusión de la normativa de protección de datos personales, siempre que los conjuntos de datos finalmente

utilizados estén agregados, anonimizados o no permitan identificar directa ni indirectamente a personas físicas.

No obstante, se tendrá en cuenta que la AEPD considera datos personales cualquier información relativa a una persona física identificada o identificable, incluyendo datos de localización o identificadores en línea cuando permitan identificar a una persona. Por este motivo, antes de utilizar cada conjunto de datos se revisará si contiene información personal.

Si durante el desarrollo del trabajo se detectase que algún conjunto de datos pudiera permitir la identificación de personas, se descartará su uso o se sustituirá por una fuente agregada o anonimizada. En caso de no poder sustituirse, sería necesario aplicar el RGPD y la Ley Orgánica 3/2018, que adapta el ordenamiento jurídico español al Reglamento (UE) 2016/679 y se aplica al tratamiento automatizado o no automatizado de datos personales.

A.3. Finalidad del tratamiento

La finalidad del tratamiento de los datos será exclusivamente académica. Los datos se utilizarán para validar el funcionamiento de la arquitectura propuesta, comprobar su capacidad para integrar y procesar información de movilidad urbana, aplicar técnicas de análisis predictivo y representar los resultados mediante visualizaciones interactivas.

No se utilizarán los datos para tomar decisiones automatizadas con efectos sobre personas, elaborar perfiles individuales, realizar seguimiento de usuarios ni identificar patrones de comportamiento personal.

A.4. Medidas aplicadas

Durante el desarrollo del TFM se aplicarán las siguientes medidas:

- Utilizar únicamente datos abiertos, agregados o anonimizados.
- Evitar el uso de identificadores personales o información individualizada.
- No combinar fuentes con el objetivo de reconstruir trayectorias individuales.
- Documentar el origen de los datos utilizados.
- Citar correctamente las fuentes de información.

- Mantener una copia de los datos originales y otra de los datos transformados.
- Utilizar los datos únicamente con finalidad académica.
- Eliminar variables que no sean necesarias para los objetivos del análisis.

Estas medidas siguen un enfoque preventivo basado en la minimización del riesgo y en el uso responsable de la información.

A.5. Reutilización de datos abiertos

Al tratarse de datos procedentes previsiblemente de portales institucionales, también se respetará la normativa sobre reutilización de información del sector público. La Ley 37/2007 regula la reutilización de documentos elaborados o custodiados por administraciones y organismos del sector público.

En particular, se tendrán en cuenta las condiciones generales de reutilización, como citar la fuente, no desnaturalizar el sentido de la información, mencionar la fecha de actualización cuando esté disponible y no revertir procedimientos de disociación mediante la combinación con otras fuentes.

A.6. Conclusión

En conclusión, el TFM no prevé el tratamiento de datos personales identificados o identificables. La validación de la arquitectura se realizará preferentemente con datos abiertos, agregados o anonimizados, por lo que el tratamiento principal quedaría fuera del ámbito de aplicación del RGPD.

Aun así, se aplicarán buenas prácticas de privacidad, minimización, documentación y reutilización responsable de la información. Si durante el desarrollo se identificase algún conjunto de datos con riesgo, se descartará o se sustituirá por una fuente anonimizada. De esta forma, el trabajo garantiza un tratamiento prudente y responsable de los datos utilizados.

Anexo B. Estructura de carpetas del proyecto

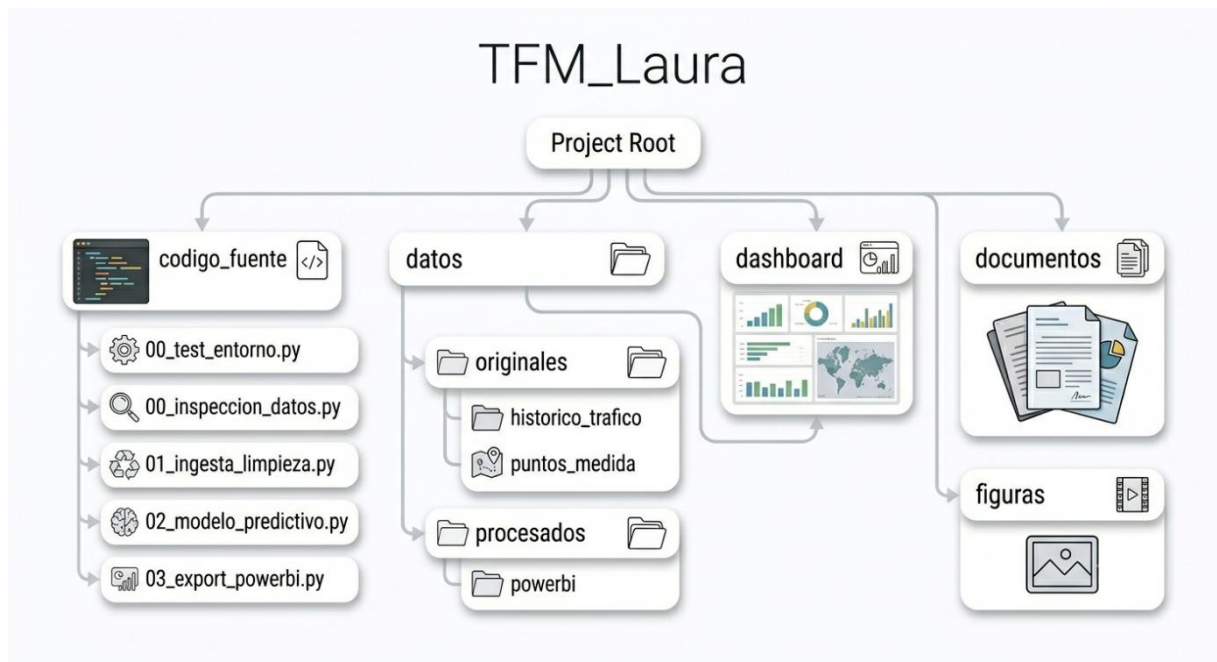


Figura 4. Estructura de carpetas del proyecto

Anexo C. Script de ingesta y limpieza

```

from pathlib import Path
import pandas as pd
import numpy as np

# =====
# CONFIGURACIÓN GENERAL
# =====

BASE = Path(__file__).resolve().parent.parent
RUTA_HISTORICO = BASE / "datos" / "originales" / "historico_trafico"
RUTA_PUNTOS = BASE / "datos" / "originales" / "puntos_medida"
RUTA_PROCESADOS = BASE / "datos" / "procesados" / "definitivos"
RUTA_PROCESADOS.mkdir(parents=True, exist_ok=True)

COLUMNAS_HISTORICO = [
    "id",
    "fecha",
    "tipo_elem",
    "intensidad",
    "ocupacion",
    "carga",
    "vmed",
    "error",

```

```

"periodo_integracion"
]

CHUNKSIZE = 500_000
MAX_PUNTOS = 50
MIN_REGISTROS_HORA = 4

# =====
# FUNCIONES AUXILIARES
# =====

#Función que lee el Excel de coordenadas y prepara la información geográfica de los sensores
def leer_puntos_medida():
    """
    Lee el archivo Excel de puntos de medida y devuelve una tabla limpia
    con id, nombre, distrito, longitud y latitud.
    """
    archivos = sorted(RUTA_PUNTOS.glob("*.xlsx")) + sorted(RUTA_PUNTOS.glob("*.xls")) #se
    ordenan los archivos

    if not archivos:
        raise FileNotFoundError(f"No se ha encontrado ningún archivo Excel en {RUTA_PUNTOS}")

    archivo = archivos[0]
    print(f"Leyendo puntos de medida: {archivo.name}")
    puntos = pd.read_excel(archivo)

    puntos.columns = (
        puntos.columns
        .str.strip() #espacios en blanco
        .str.lower() #minúsculas
        .str.replace(" ", "_") #espacios por guiones
    )

    columnas_necesarias = ["id", "tipo_elem", "distrito", "nombre", "longitud", "latitud"]
    columnas_existentes = [c for c in columnas_necesarias if c in puntos.columns]
    puntos = puntos[columnas_existentes].copy() #copia las columnas existentes

    puntos["id"] = pd.to_numeric(puntos["id"], errors="coerce") #convierte id a número
    puntos = puntos.dropna(subset=["id"]) #elimina filas sin id
    puntos = puntos.drop_duplicates(subset=["id"]) #elimina duplicados del sensor
    puntos["id"] = puntos["id"].astype(int) #convierte id a entero

    if "longitud" in puntos.columns: #si existe longitud en las columnas, convierte a número
        puntos["longitud"] = pd.to_numeric(puntos["longitud"], errors="coerce")

    if "latitud" in puntos.columns: #si existe latitud en las columnas, convierte a número

```

```

puntos["latitud"] = pd.to_numeric(puntos["latitud"], errors="coerce")

puntos.to_csv(RUTA_PROCESADOS / "puntos_medida_limpios.csv", index=False, sep=";")
#guarda la tabla limpia

print(f"Puntos de medida limpios: {len(puntos)}")
return puntos

#Función que escoge los 50 sensores con más registros marcados como válidos
def seleccionar_top_puntos(archivo_referencia):
    """
    Selecciona los puntos con más registros válidos en el primer archivo.
    """
    print(f"Seleccionando los {MAX_PUNTOS} puntos con más registros usando:
    {archivo_referencia.name}")

    conteos = {} #guarda los registros válidos de cada sensor

    for chunk in pd.read_csv( #lee el CSV por bloques de 500.000 filas
        archivo_referencia,
        sep=";",
        encoding="utf-8",
        usecols=["id", "error"], #limita la lectura a id y error
        chunksize=CHUNKSIZE
    ):
        chunk = chunk[chunk["error"] #filtra registros cuyo indicador de error es N
        .astype(str) #convierte valores a texto
        .str.strip() #elimina espacios
        .str.upper() == "N"] #mayúsculas
        vc = pd.to_numeric(chunk["id"], errors="coerce").dropna().astype(int).value_counts()

        for id_punto, n in vc.items(): #recorre los conteos del bloque
            conteos[id_punto] = conteos.get(id_punto, 0) + int(n) #recupera el acumulado anterior
            y suma los registros del bloque actual

    serie = pd.Series(conteos, dtype="int64").sort_values(ascending=False) #ordena de mayor
    a menor
    top_ids = serie.head(MAX_PUNTOS).index.astype(int).tolist() #selecciona los 50 primeros

    print(f"Puntos seleccionados: {len(top_ids)}")
    print(f"Primeros puntos seleccionados: {top_ids[:10]}")

    pd.DataFrame({ #crea y guarda un csv con los sensores seleccionados y sus conteos
        "id": top_ids,
        "registros_archivo_referencia": serie.head(MAX_PUNTOS).values
    }).to_csv(RUTA_PROCESADOS / "puntos_seleccionados_modelo.csv", index=False, sep=";")

```

```
return top_ids
```

```
#Función que recibe un csv y devuelve un resumen de calidad
def procesar_archivo_historico(archivo, ids_seleccionados):
```

```
    """
```

```
    Procesa un CSV mensual por bloques:
```

- filtra errores
- convierte fechas
- convierte numéricos
- filtra intensidades negativas
- agrega por hora y punto de medida

```
    """
```

```
    print(f"\nProcesando archivo: {archivo.name}")
```

```
    grupos_archivo = [] #lista que guarda los bloques filtrados
```

```
    registros_leidos = 0 #cuenta las filas originales
```

```
    registros_validos = 0 #cuenta las filas que superen los filtros
```

```
    for chunk in pd.read_csv( #lee el archivo por bloques
```

```
        archivo,
```

```
        sep=";",
```

```
        encoding="utf-8",
```

```
        usecols=COLUMNAS_HISTORICO,
```

```
        chunksize=CHUNKSIZE
```

```
    ):

```

```
        registros_leidos += len(chunk) #devuelve el número de filas por bloque y lo suma a lo
        acumulado
```

```
        chunk = chunk[chunk["id"].isin(ids_seleccionados)].copy() #Comprueba si el id está entre
        los sensores seleccionados
```

```
        chunk = chunk[chunk["error"].astype(str).str.strip().str.upper() == "N"].copy() #Filtra
        registros sin error
```

```
        chunk["fecha"] = pd.to_datetime(chunk["fecha"], errors="coerce") #Convierte la fecha a
        tipo datetime
```

```
        for col in ["intensidad", "ocupacion", "carga", "vmed", "periodo_integracion"]: #Convierte
        en numéricas las variables
```

```
            chunk[col] = pd.to_numeric(chunk[col], errors="coerce")
```

```
        chunk = chunk.dropna(subset=["id", "fecha", "intensidad"]) #Elimina filas sin id, fecha e
        intensidad
```

```
        chunk = chunk[chunk["intensidad"] >= 0].copy() #Descarta las intensidades negativas
```

```
        registros_validos += len(chunk) #Suma al contador los registros que han pasado el filtro
```

```

    grupos_archivo.append(chunk[["id", "fecha", "intensidad", "ocupacion",
                                "carga", "vmed", "periodo_integracion"]].copy()) #Añade a la lista de bloques los
necesarios

if not grupos_archivo:
    return pd.DataFrame(), {"archivo": archivo.name, "registros_leidos": registros_leidos,
                            "registros_validos": 0, "registros_horarios": 0} #Devuelve un dataframe vacío si no se ha
añadido ningún bloque

datos = pd.concat(grupos_archivo, ignore_index=True) #Une los bloques filtrados

duplicados = int(datos.duplicated(["id", "fecha"]).sum()) #Elimina duplicados de sensor y
fecha
if duplicados: #Si existen duplicados el programa se detiene
    raise ValueError(f"{archivo.name}: {duplicados} fechas duplicadas por sensor; "
                    "revisar antes de agregar.")

datos["fecha_hora"] = datos["fecha"].dt.floor("h") #Redondea cada hora a su comienzo

grupo = datos.groupby(["id", "fecha_hora"], as_index=False).agg( #Agrupar las filas que
pertenecen al mismo sensor y misma hora
    intensidad_horaria=("intensidad", "mean"), #Se definen agregaciones
    ocupacion_media=("ocupacion", "mean"),
    carga_media=("carga", "mean"),
    velocidad_media=("vmed", "mean"),
    periodo_integracion_medio=("periodo_integracion", "mean"),
    registros_15min=("intensidad", "count")
)

cuartos = datos.groupby(["id", "fecha_hora"])["fecha"].nunique() #Cuenta los timestamps
distintos
if (cuartos > 4).any(): #Sale error si alguna hora tiene más de 4 timestamps distintos
    raise ValueError(f"{archivo.name}: más de cuatro timestamps por hora; revisar
frecuencia.")

incompletas = int((grupo["registros_15min"] != MIN_REGISTROS_HORA).sum()) #Cuenta las
horas que no tienen cuatro registros
grupo = grupo[grupo["registros_15min"] == MIN_REGISTROS_HORA].copy() #Conserva las
horas completas

minutos = datos["fecha"].dt.minute #Comprueba que el minuto está entre 0, 15, 30 y 45
if not minutos.isin([0, 15, 30, 45]).all():
    raise ValueError(f"{archivo.name}: timestamps fuera de los cuartos esperados.")

print(f"Registros leídos: {registros_leidos}")
print(f"Registros válidos tras limpieza: {registros_validos}")
print(f"Registros horarios generados: {len(grupo)}")

```

```

resumen = { #Diccionario con las estadísticas del mes
    "archivo": archivo.name,
    "registros_leidos": registros_leidos,
    "registros_validos": registros_validos,
    "registros_horarios": len(grupo),
    "horas_incompletas_excluidas": incompletas
}
return grupo, resumen

# =====
# PROCESO PRINCIPAL
# =====

def main():
    print("=" * 100)
    print("INICIO DEL PROCESO DE INGESTA Y LIMPIEZA")
    print("=" * 100)

    archivos_historico = sorted(RUTA_HISTORICO.glob("*.csv")) #Ordena todos los archivos

    if not archivos_historico: #Si no los encuentra, sale error
        raise FileNotFoundError(f"No se han encontrado CSV en {RUTA_HISTORICO}")
    print(f"Archivos históricos encontrados: {len(archivos_historico)}")

    puntos = leer_puntos_medida() #Utiliza el primer csv ordenado
    ids_seleccionados = seleccionar_top_puntos(archivos_historico[0])

    resultados = []
    resmenes = []

    for archivo in archivos_historico: #Recorre los archivos procesados y los muestra
        print("Procesando:", archivo.name, flush=True)
        mes, resumen = procesar_archivo_historico(archivo, ids_seleccionados)

        if not mes.empty: #Si no está vacío el dataframe, lo añade a la lista de resultados
            resultados.append(mes)
            resmenes.append(resumen)
            print(resumen, flush=True)

    if not resultados: #Si no hay datos, da error
        raise ValueError("No se han generado registros válidos.")

    print("\nUniando todos los meses procesados...")

    trafico = pd.concat(resultados, ignore_index=True) #Une los dataframes en una tabla

```

```

if trafico.duplicated(["id", "fecha_hora"]).any(): #Comprueba que no hayan mismas horas
    en un mismo sensor
    raise ValueError("Existen horas duplicadas entre archivos mensuales.")

trafico["año"] = trafico["fecha_hora"].dt.year #Se crean variables temporales
trafico["mes"] = trafico["fecha_hora"].dt.month
trafico["dia"] = trafico["fecha_hora"].dt.day
trafico["hora"] = trafico["fecha_hora"].dt.hour
trafico["dia_semana"] = trafico["fecha_hora"].dt.dayofweek
trafico["es_fin_semana"] = trafico["dia_semana"].isin([5, 6]).astype(int)

trafico = trafico.merge( #Se unen las coordenadas mediante la unión
    puntos,
    on="id",
    how="left",
    validate="many_to_one"
)

trafico = trafico.sort_values(["id", "fecha_hora"]) #Ordena el dataset por sensor y por fecha

trafico.to_csv(RUTA_PROCESADOS / "trafico_limpio.csv", index=False, sep=";") # Exporta el
dataset limpio

pd.DataFrame(resumenes).to_csv(RUTA_PROCESADOS / "resumen_limpieza.csv",
index=False, sep=";") #Convierte resúmenes en una tabla

cobertura = trafico.groupby(["id", "año",
"mes"]).size().rename("horas_observadas").reset_index() #Agrupar los datos
cobertura.to_csv(RUTA_PROCESADOS / "cobertura_mensual.csv", index=False, sep=";")
print("Dataset generado:", len(trafico), "filas")
print("Salida:", RUTA_PROCESADOS)

print("\n" + "=" * 100)
print("PROCESO FINALIZADO")
print("=" * 100)

if __name__ == "__main__":
    main()

```

Anexo D. Script de modelado predictivo

```

from pathlib import Path
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

```

```

# =====
# CONFIGURACIÓN
# =====

BASE = Path(__file__).resolve().parent.parent
RUTA_PROCESADOS = BASE / "datos" / "procesados" / "definitivos"

ARCHIVO_ENTRADA = RUTA_PROCESADOS / "trafico_limpio.csv"
ARCHIVO_MODELO = RUTA_PROCESADOS / "trafico_modelo.csv"
ARCHIVO_PREDICCIONES = RUTA_PROCESADOS / "predicciones.csv"
ARCHIVO_METRICAS = RUTA_PROCESADOS / "metricas_modelo.csv"

print("=" * 100)
print("INICIO DEL MODELO PREDICTIVO")
print("=" * 100)

#Entrenamiento hasta septiembre de 2023
FECHA_VALIDACION = pd.Timestamp("2023-10-01") #Validación de octubre a diciembre de 2023
FECHA_PRUEBA = pd.Timestamp("2024-01-01") #Prueba desde enero de 2024

VARIABLES_OPERATIVAS = ["ocupacion_media", "carga_media", "velocidad_media",
"registros_15min"] #Variable para predecir el tráfico

# =====
# PREPARACIÓN DE LOS DATOS
# =====

#Función para los datos para que cada variable predictiva respete el tiempo y no se mezclen
sensores
def preparar_datos(df):
    df = df.copy()

    df["fecha_hora"] = pd.to_datetime(df["fecha_hora"], errors="coerce")

    df = df.dropna(subset=["id", "fecha_hora", "intensidad_horaria"]) #Elimina las filas sin id,
fecha e intensidad

    df["id"] = pd.to_numeric(df["id"], errors="coerce") #Convierte a numérico
    df = df.dropna(subset=["id"]) #Elimina los que no son válidos
    df["id"] = df["id"].astype(int) #Los transforma en enteros

    if df.duplicated(["id", "fecha_hora"]).any(): #Comprueba duplicados del mismo sensor a la
misma hora
        raise ValueError("Existen duplicados id-hora. Revisa la ingesta.")

```

```

df = df.sort_values(["id", "fecha_hora"]) #Ordena por sensor y por fecha

grupos = []
for id_punto, grupo in df.groupby("id", sort=False): #Recorre los datos sensor por sensor
    grupo = grupo.set_index("fecha_hora")
    calendario = pd.date_range(grupo.index.min(), grupo.index.max(), freq="h") #Genera
todas las horas del sensor
    grupo = grupo.reindex(calendario) #Alinea los datos con el calendario completo
    grupo.index.name = "fecha_hora" #Da nombre al índice
    grupo["id"] = id_punto #Asigna el id del sensor
    grupos.append(grupo.reset_index()) #Añade la fecha convertida en columna a la lista

df = pd.concat(grupos, ignore_index=True) #Concatena los calendarios de sensores en una
tabla
df = df.sort_values(["id", "fecha_hora"]).reset_index(drop=True)

y = df.groupby("id")["intensidad_horaria"] #Crea un objeto agrupado por sensor que tiene
la columna de intensidad

df["intensidad_lag_1h"] = y.shift(1) #Desplaza los valores una fila abajo de cada sensor
df["intensidad_lag_24h"] = y.shift(24)

for ventana in [3, 24]: #Recorre dos tamaños de ventana: 3 y 24 horas
    df[f"media_movil_{ventana}h"] = (
        df.groupby("id")["intensidad_horaria"] #Separa las series por sensor
        .transform(lambda s: s #Define una función que recibe la serie de intensidad de un
sensor
        .shift(1) #Desplaza la serie una hora hacia el pasado
        .rolling(ventana, min_periods=ventana) #Crea ventanas cuyos datos estén disponibles
        .mean()) #Calcula la media de cada ventana
    )

for col in VARIABLES_OPERATIVAS:
    if col in df: #Comprueba si la columna existe en el dataframe
        df[f"{col}_lag_1h"] = df.groupby("id")[col].shift(1) #Crea columnas con una versión con
sufijo de una hora

df["año"] = df["fecha_hora"].dt.year
df["mes"] = df["fecha_hora"].dt.month
df["dia"] = df["fecha_hora"].dt.day
df["hora"] = df["fecha_hora"].dt.hour
df["dia_semana"] = df["fecha_hora"].dt.dayofweek
df["es_fin_semana"] = df["dia_semana"].isin([5, 6]).astype(int) #Convierte la condición de
sábado o domingo en 1 o 0

features = ["id", "mes", "dia", "hora", "dia_semana", "es_fin_semana", #Lista columnas
entrada Random Forest

```

```

    "intensidad_lag_1h", "intensidad_lag_24h",
    "media_movil_3h", "media_movil_24h"]

    features += [f"{c}_lag_1h" for c in VARIABLES_OPERATIVAS if c in df] #Versiones retardadas
    de variables operativas
    df_modelo = df.dropna(subset=features + ["intensidad_horaria"]).copy() #Elimina filas
    return df_modelo, features, df

# =====
# CÁLCULO DE MÉTRICAS
# =====

#Función para el cálculo de métricas
def calcular_metricas(y_real, y_pred, nombre, conjunto): #Recibe valores
    return {"modelo": nombre, "conjunto": conjunto, #Devuelve un diccionario con las métricas
    y el número de observaciones
        "MAE": mean_absolute_error(y_real, y_pred),
        "RMSE": np.sqrt(mean_squared_error(y_real, y_pred)),
        "R2": r2_score(y_real, y_pred), "n": len(y_real)}

def main():
    print("=" * 80)
    print("INICIO DEL MODELO PREDICTIVO")
    print("=" * 80)

    df = pd.read_csv(ARCHIVO_ENTRADA, sep=";") #Lee el dataset corregido
    df_modelo, features, calendario = preparar_datos(df)

    print("Registros originales:", len(df))
    print("Filas calendario:", len(calendario))
    print("Registros disponibles para modelado:", len(df_modelo))
    print("Variables:", features)

    train = df_modelo[df_modelo["fecha_hora"] < FECHA_VALIDACION].copy() #Selecciona filas
    antes al 1 octubre

    val = df_modelo[(df_modelo["fecha_hora"] >= FECHA_VALIDACION) & #Filas desde 1
    octubre a 31 diciembre
        (df_modelo["fecha_hora"] < FECHA_PRUEBA)].copy()

    test = df_modelo[df_modelo["fecha_hora"] >= FECHA_PRUEBA].copy() #Filas desde el 1 de
    enero 2024

    if train.empty or val.empty or test.empty: #Comprueba que ninguna partición esté vacía
        raise ValueError("Una partición temporal está vacía.")

    print("Entrenamiento:", len(train), "Validación:", len(val), "Prueba:", len(test))

```

```

target = "intensidad_horaria" #Define la columna que quiero predecir

# =====
# CREACIÓN Y ENTRENAMIENTO RANDOM FOREST
# =====

modelo_rf = RandomForestRegressor( #Crea el modelo con los parámetros
    n_estimators=100, max_depth=14, min_samples_leaf=5,
    random_state=42, n_jobs=-1)
modelo_rf.fit(train[features], train[target]) #Entrena el modelo

pred_val = modelo_rf.predict(val[features]) #Aplica el modelo al conjunto de validación
validacion = pd.DataFrame([ #Crea una tabla con las métricas de tres modelos
    calcular_metricas(val[target], val["intensidad_lag_1h"], "Persistencia", "validacion"),
    calcular_metricas(val[target], val["intensidad_lag_24h"], "Baseline lag 24h", "validacion"),
    calcular_metricas(val[target], pred_val, "Random Forest Regressor", "validacion")
])
validacion.to_csv(RUTA_PROCESADOS / "metricas_validacion.csv", index=False, sep=";")

train_final = pd.concat([train, val], ignore_index=True)
modelo_rf.fit(train_final[features], train_final[target]) #Vuelve a entrenar el modelo
pred_rf = modelo_rf.predict(test[features]) #Genera las predicciones sobre 2024

pred_rf = np.maximum(pred_rf, 0) #Compara cada predicción con 0 y conserva el mayor

metricas = pd.DataFrame([ #Calcula las métricas definitivas de los tres modelos
    calcular_metricas(test[target], test["intensidad_lag_1h"], "Persistencia", "prueba"),
    calcular_metricas(test[target], test["intensidad_lag_24h"], "Baseline lag 24h", "prueba"),
    calcular_metricas(test[target], pred_rf, "Random Forest Regressor", "prueba")
])
metricas.to_csv(ARCHIVO_METRICAS, index=False, sep=";")
print(metricas.to_string(index=False))

predicciones = test.copy()
predicciones["prediccion_persistencia"] = test["intensidad_lag_1h"] #Añade tres columnas
con predicciones
predicciones["prediccion_baseline"] = test["intensidad_lag_24h"]
predicciones["prediccion_rf"] = pred_rf

predicciones["error_rf"] = predicciones[target] - predicciones["prediccion_rf"] #Calcula el
error con signo
predicciones["error_abs_rf"] = predicciones["error_rf"].abs() #Calcula el error absoluto

columnas = ["id", "fecha_hora", "año", "mes", "dia", "hora", "dia_semana", #Valores para
PowerBi
    "es_fin_semana", target, "prediccion_persistencia",
    "prediccion_baseline", "prediccion_rf", "error_rf", "error_abs_rf",

```

```

"latitud", "longitud", "nombre", "distrito"]

predicciones[[c for c in columnas if c in predicciones]].to_csv(
    ARCHIVO_PREDICCIONES, index=False, sep=";")

df_modelo.to_csv(ARCHIVO_MODELO, index=False, sep=";")

pd.DataFrame({"variable": features, "importancia": modelo_rf.feature_importances_}
    ).sort_values("importancia", ascending=False).to_csv(
    RUTA_PROCESADOS / "importancia_variables.csv", index=False, sep=";")

pd.DataFrame([ #Crea un csv con el número de registros de cada partición
    {"conjunto": "entrenamiento", "registros": len(train)},
    {"conjunto": "validacion", "registros": len(val)},
    {"conjunto": "entrenamiento_final", "registros": len(train_final)},
    {"conjunto": "prueba", "registros": len(test)}
]).to_csv(RUTA_PROCESADOS / "resumen_modelo.csv", index=False, sep=";")
print("Resultados guardados en:", RUTA_PROCESADOS)

if __name__ == "__main__":
    main()

```

Anexo E. Script de exportación a Power BI

```

from pathlib import Path
import pandas as pd

# =====
# CONFIGURACIÓN
# =====

BASE = Path(__file__).resolve().parent.parent
RUTA_PROCESADOS = BASE / "datos" / "procesados" / "definitivos"
RUTA_POWERBI = RUTA_PROCESADOS / "powerbi"
RUTA_POWERBI.mkdir(parents=True, exist_ok=True)

# =====
# CARGA DE ARCHIVOS
# =====

print("=" * 100)
print("PREPARANDO ARCHIVOS PARA POWER BI")
print("=" * 100)

def main():

```

```

trafico = pd.read_csv(RUTA_PROCESADOS / "trafico_limpio.csv", sep=";")
predicciones = pd.read_csv(RUTA_PROCESADOS / "predicciones.csv", sep=";")
metricas = pd.read_csv(RUTA_PROCESADOS / "metricas_modelo.csv", sep=";")
importancias = pd.read_csv(RUTA_PROCESADOS / "importancia_variables.csv", sep=";")
resumen_limpieza = pd.read_csv(RUTA_PROCESADOS / "resumen_limpieza.csv", sep=";")
puntos = pd.read_csv(RUTA_PROCESADOS / "puntos_medida_limpios.csv", sep=";")
resumen_modelo = pd.read_csv(RUTA_PROCESADOS / "resumen_modelo.csv", sep=";")

# =====
# PREPARACIÓN DE FECHAS
# =====

trafico["fecha_hora"] = pd.to_datetime(trafico["fecha_hora"]) #Convierte de texto a
datetime
predicciones["fecha_hora"] = pd.to_datetime(predicciones["fecha_hora"])

trafico["fecha"] = trafico["fecha_hora"].dt.date #Crea una columna fecha sin la hora
predicciones["fecha"] = predicciones["fecha_hora"].dt.date

# =====
# TABLA RESUMEN GENERAL
# =====

resumen_general = pd.DataFrame([ #Dataframe a partir de una lista de diccionarios
    {"indicador": "Registros dataset limpio", "valor": len(trafico)}, #Cuenta las filas del dataset
    {"indicador": "Puntos de medida analizados", "valor": trafico["id"].nunique()}, #Cuenta los
id distintos de sensor
    {"indicador": "Fecha inicial", "valor": str(trafico["fecha_hora"].min())}, #Obtiene primer y
última fecha y lo convierte a texto
    {"indicador": "Fecha final", "valor": str(trafico["fecha_hora"].max())},
    {"indicador": "Registros de entrenamiento 2023",
"valor": resumen_modelo.loc[resumen_modelo["conjunto"] == "entrenamiento_final",
#Crea un filtro que localiza la fila del entrenamiento
"registros"].iloc[0]}, #Extrae el primer valor del resultado
    {"indicador": "Registros de prueba 2024", #Igual, pero para el conjunto de prueba 2024
"valor": resumen_modelo.loc[resumen_modelo["conjunto"] == "prueba",
"registros"].iloc[0]}
])

# =====
# TABLA DE CALIDAD DE DATOS
# =====

calidad_datos = pd.DataFrame({ #Crea una tabla que analiza los nulos en cada columna
"columna": trafico.columns,
"nulos": trafico.isna().sum().values,
"porcentaje_nulos": trafico.isna().sum().values / len(trafico) * 100

```

```

})

# =====
# TABLA TEMPORAL AGREGADA
# =====

trafico_temporal = trafico.groupby(
    ["año", "mes", "dia_semana", "hora"], as_index=False
).agg( #Calcula indicadores
    intensidad_media=("intensidad_horaria", "mean"),
    intensidad_total=("intensidad_horaria", "sum"),
    ocupacion_media=("ocupacion_media", "mean"),
    velocidad_media=("velocidad_media", "mean"),
    registros=("id", "count"))

# =====
# TABLA POR PUNTO DE MEDIDA
# =====

trafico_puntos = trafico.groupby( #Tabla que agrupa los datos de cada sensor
    ["id", "nombre", "distrito", "latitud", "longitud"], as_index=False
).agg(
    intensidad_media=("intensidad_horaria", "mean"),
    intensidad_total=("intensidad_horaria", "sum"),
    ocupacion_media=("ocupacion_media", "mean"),
    velocidad_media=("velocidad_media", "mean"),
    registros=("fecha_hora", "count"))

# =====
# TABLA DE PREDICCIONES REDUCIDA
# =====

columnas_pred = [ #Columnas que se exportan de la tabla de predicciones
    "id", "fecha_hora", "fecha", "año", "mes", "dia", "hora",
    "dia_semana", "es_fin_semana", "intensidad_horaria",
    "prediccion_persistencia", "prediccion_baseline", #Predicción hora anterior, predicción
    mismo hora día anterior
    "prediccion_rf", "error_rf", "error_abs_rf", #Predicción generada por Random Forest
    "latitud", "longitud", "nombre", "distrito"
]

predicciones[[c for c in columnas_pred if c in predicciones]]

# =====
# EXPORTACIÓN
# =====

```

```

tablas = {
    "trafico_powerbi.csv": trafico,
    "trafico_temporal_powerbi.csv": trafico_temporal,
    "trafico_puntos_powerbi.csv": trafico_puntos,
    "predicciones_powerbi.csv": predicciones[[c for c in columnas_pred if c in predicciones]],
    "metricas_powerbi.csv": metricas,
    "importancia_variables_powerbi.csv": importancias,
    "resumen_limpieza_powerbi.csv": resumen_limpieza,
    "resumen_general_powerbi.csv": resumen_general,
    "calidad_datos_powerbi.csv": calidad_datos,
    "puntos_medida_powerbi.csv": puntos
}
for nombre, tabla in tablas.items(): #Bucle de exportación
    tabla.to_csv(RUTA_POWERBI / nombre, index=False, sep=";", #Guarda cada dataframe
    como csv
                encoding="utf-8-sig", decimal=",")
    print("Exportaciones generadas en:", RUTA_POWERBI)

if __name__ == "__main__":
    main()

```

Anexo F. Imágenes del Dashboard en Power BI

Página 1 – Portada



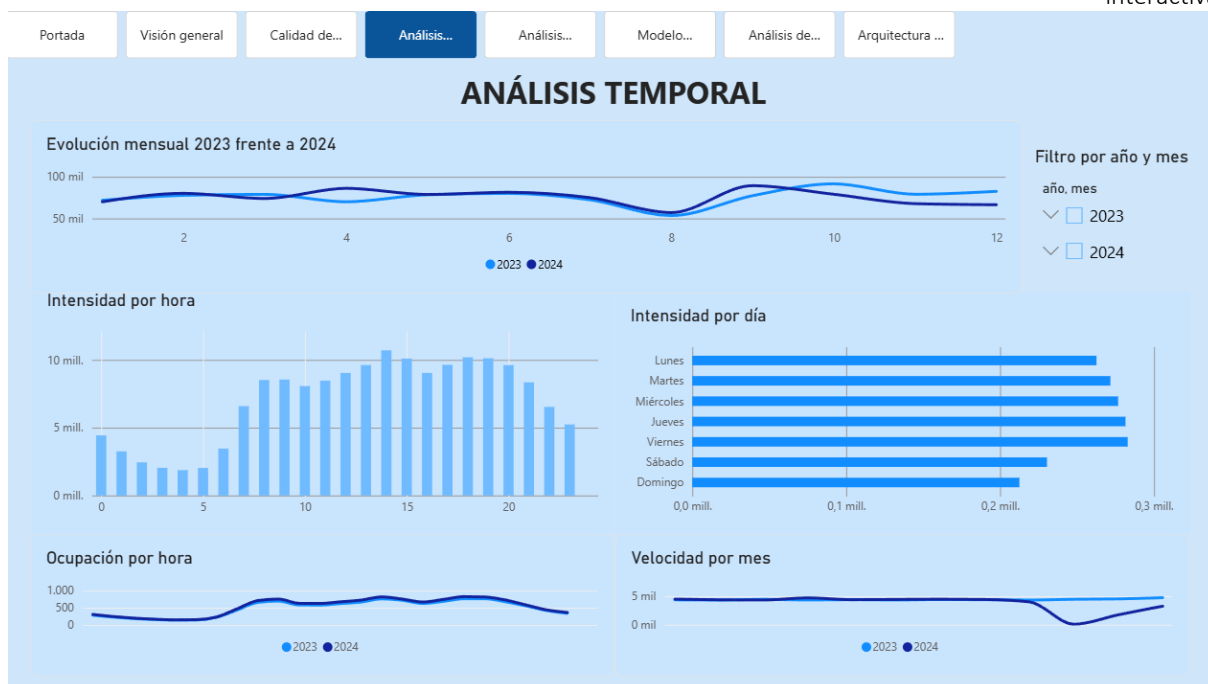
Página 2 – Visión general del tráfico



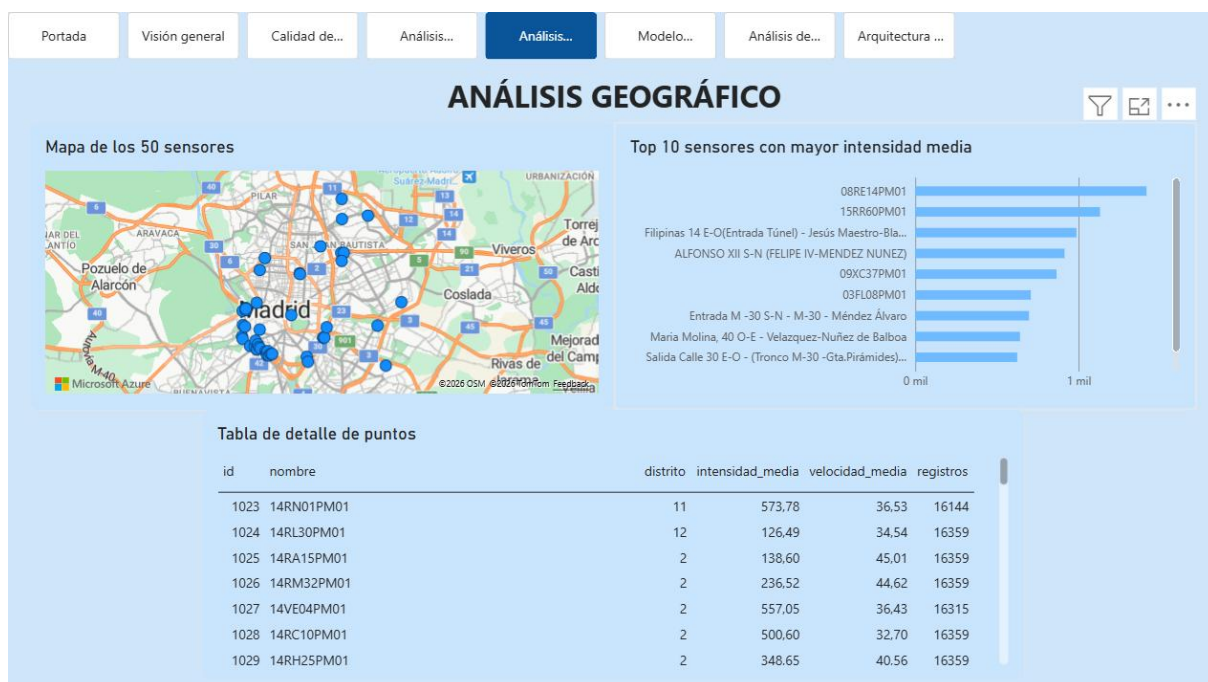
Página 3 – Calidad de los datos



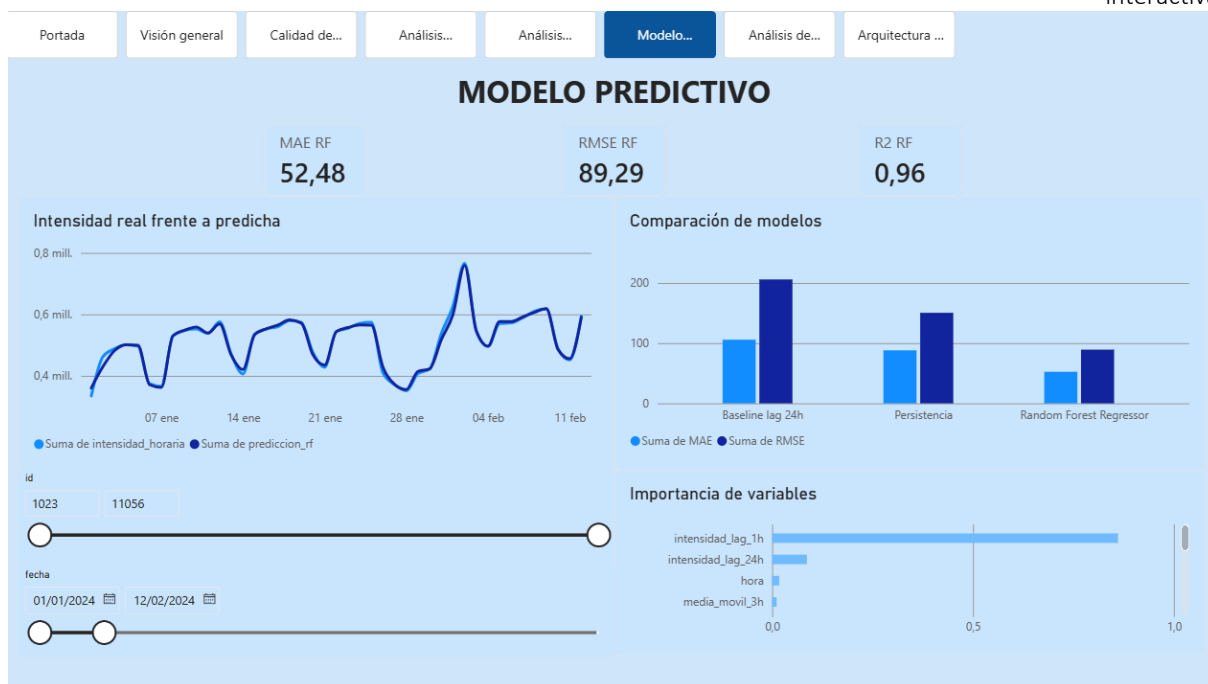
Página 4 – Análisis temporal



Página 5 – Análisis geográfico



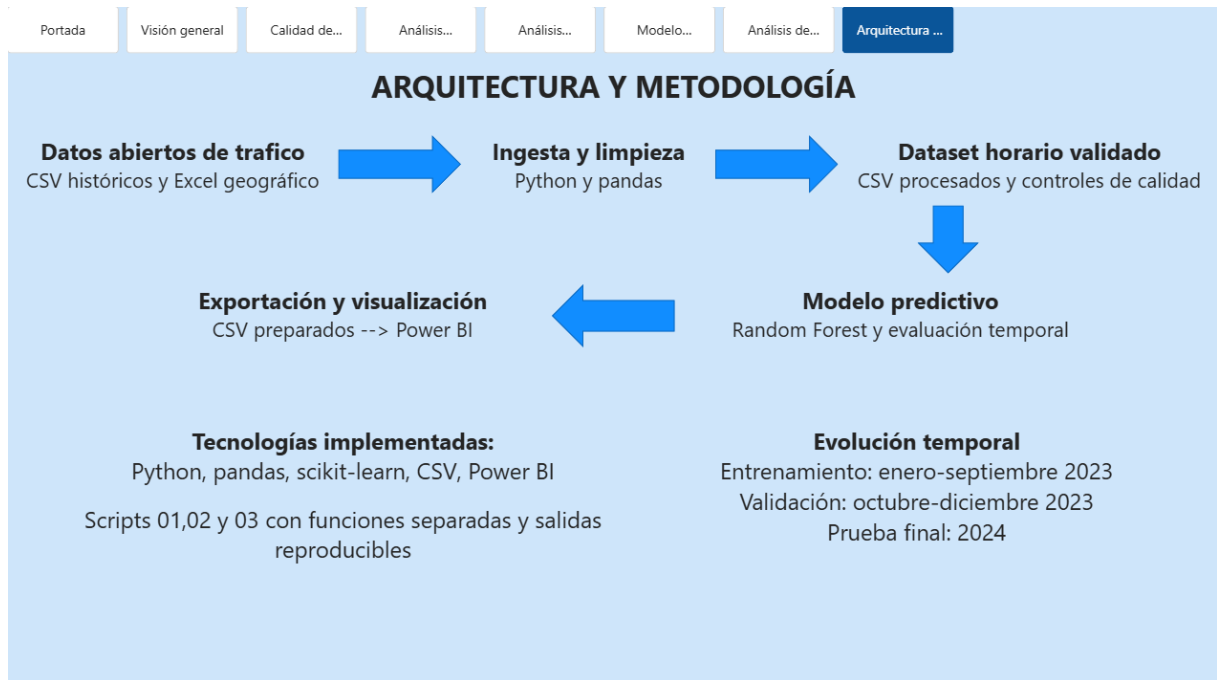
Página 6 – Modelo predictivo



Página 7 – Análisis de errores



Página 8 – Arquitectura y metodología



Anexo G. Declaración de uso de herramientas de IA

Para la elaboración de este Trabajo Fin de Máster se ha utilizado ChatGPT (OpenAI) como herramienta de apoyo en distintas fases del trabajo.

Su utilización se ha centrado principalmente en las siguientes tareas:

- Ordenación y estructuración de la memoria, dividiendo el trabajo por apartados y subapartados para facilitar su desarrollo.
- Revisión y comentario del código en Python, como apoyo para comprender, corregir y reorganizar scripts relacionados con la ingesta, limpieza, modelado predictivo y exportación de resultados.
- Apoyo en la detección de errores, incoherencias o mejoras tanto en el código como en la redacción del documento.
- Apoyo en la elaboración de imágenes y esquemas explicativos, especialmente relacionados con la organización del proyecto y la arquitectura desarrollada.
- Revisión formal de textos, incluyendo sugerencias de redacción, reorganización de contenido y mejora de la claridad expositiva.

La herramienta no se ha utilizado como sustituto del trabajo personal ni como fuente académica primaria. Todas las decisiones metodológicas, la implementación del código, la ejecución del pipeline, la validación de resultados, la selección de referencias y la redacción final del documento han sido revisadas y comprobadas por la autora.

Asimismo, cualquier información teórica, normativa o técnica incorporada al trabajo ha sido contrastada con fuentes académicas, documentación oficial o bibliografía especializada, citadas de forma independiente en la memoria. La responsabilidad final sobre el contenido del TFM corresponde íntegramente a la autora.